

Long-Context Modeling

Context Compression, Hierarchical Attention, and Recurrent Memory

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 7, 2026

1. Motivation: what long context is for, the three costs that grow with length, and three families of techniques
2. Effective context: lost in the middle, needle tests, RULER, BABILong and NoLiMa, and extending the window
3. Prompt compression: token dropping, question-aware and learned compression, gist tokens, and memory slots
4. KV-cache compression: attention sinks, heavy hitters, SnapKV, and per-layer budgets
5. Hierarchical attention: HAN, two-level document models, Longformer, BigBird, and H-Transformer-1D
6. Block-sparse attention: landmark tokens, MoBA, Native Sparse Attention, and DeepSeek Sparse Attention
7. Segment recurrence: Transformer-XL, Compressive and Memorizing Transformers, and the Recurrent Memory Transformer
8. Linear-attention memory: linear attention as an RNN, the delta rule, Infini-attention, Titans, and hybrids
9. Choosing a technique: retrieval or a longer window, routing between them, and a comparison
10. Summary and references

- ▶ Many real inputs do not fit in a few thousand tokens: a code repository and the issue filed against it, a contract or a book, hours of transcribed audio, and the trace an agent accumulates as tool calls pile up.
- ▶ Context windows have grown from 2K tokens to 128K and beyond. The Gemini 1.5 report goes further: **near-perfect retrieval (>99%) up to at least 10M tokens** in its experiments.
- ▶ A long window lets a model use material it was never trained on. Given a grammar manual for Kalamang, a language with fewer than 200 speakers, Gemini 1.5 translated English to Kalamang at a level similar to a person who learned from the same materials.
- ▶ The window is a capacity, and an expensive one. Three questions organise this lecture: what a long context costs, how much of it a model actually uses, and which techniques make long inputs affordable.

Gemini Team, Google, "Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context," 2024, [arXiv:2403.05530v5](https://arxiv.org/abs/2403.05530v5), abstract.

Three Costs That Grow With Length

- **Compute.** Over a causal sequence of n tokens, attention costs about $2n^2d$ FLOPs per layer and the weight matrices $2nP$ (d : model width, P : parameters per layer). Attention dominates once $n > P/d$.
- **Memory.** The KV cache holds a key and a value per token, layer and KV head:

$$\text{bytes} = 2 \cdot L \cdot h_{kv} \cdot d_h \cdot (\text{bytes per value}) \cdot n,$$

for L layers and h_{kv} KV heads of dimension d_h . It is linear in n and kept per request.

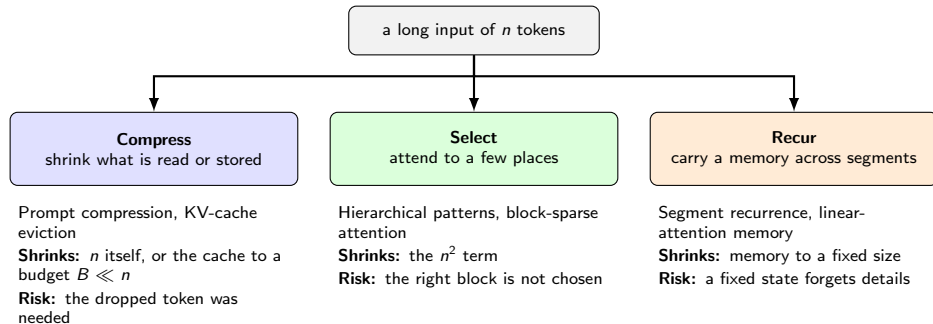
- **Use.** Positions past the training length are out of distribution (RoPE rescaling), and models under-use even trained positions.
- Llama 3 trains on long sequences last because “the compute in self-attention layers grows quadratically in the sequence length.”

Llama Team, AI @ Meta, “The Llama 3 Herd of Models,” 2024, [arXiv:2407.21783v3](https://arxiv.org/abs/2407.21783v3), Table 3 and Section 3.4.2; table values computed from that configuration.

Worked example: Llama 3.1 8B

n (tokens)	KV cache	attention / weights
8,192	1 GiB	0.15
32,768	4 GiB	0.62
131,072	16 GiB	2.46
1,048,576	128 GiB	19.7

- $L = 32$, $h_{kv} = 8$, $d_h = 128$, $d = 4096$, FFN width 14,336, bf16.
- $2 \cdot 32 \cdot 8 \cdot 128 \cdot 2 = 131,072$ bytes: 128 KiB per token, so one 128K-token request needs 16 GiB, about the size of the bf16 weights.
- $P \approx 218$ million parameters per layer, so $P/d = 53,248$ tokens.



- Each family trades exactness for cost in a different place. The lecture first measures how much context models really use, then takes the three families in turn and ends with how to choose between them and retrieval.

By the end of this lecture you should be able to:

- ▶ **Compute** the attention FLOPs and KV-cache memory of a long context for a given model, and name which of the three costs a technique reduces.
- ▶ **Explain** the gap between a claimed and an effective context length, and **choose** benchmarks that expose it.
- ▶ **Compare** prompt-compression methods and KV-cache eviction policies, and predict what each one loses.
- ▶ **Explain** hierarchical attention, from fixed local and global patterns to learned block selection, and **compute** its cost.
- ▶ **Derive** segment-level recurrence and linear-attention memory, and state what a fixed-size memory can and cannot recall.
- ▶ **Choose** between a longer window, retrieval, compression and memory for a given workload.
- ▶ **Prerequisite knowledge:** the KV cache and its quantization, GQA and MLA, FlashAttention; RoPE rescaling, sliding-window attention; RNNs, state-space models and their copying limit, mixture of experts; soft prompts, LoRA, RAG.

- ▶ A model's **claimed** context length is the longest input it accepts, n tokens. It states a capacity; how much of it the model uses has to be measured.
- ▶ RULER, a benchmark of 13 synthetic tasks, defines the **effective** context length as the longest tested length whose score still clears a pass mark:

$$n_{\text{eff}} = \max\{n \in \mathcal{N} : a(n) > \tau\},$$

where $a(n)$ is the mean accuracy over the tasks at length n , $\mathcal{N}$ the set of tested lengths (4K to 128K) and τ the pass mark.

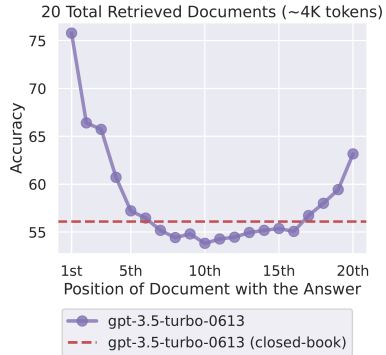
- ▶ RULER sets $\tau = 85.6\%$, the score of Llama-2-7B, a 4K-context model, at 4K. In its words, "almost all models fall below the threshold before reaching the claimed context lengths."
- ▶ n_{eff} belongs to a benchmark and a threshold; quote both with the number.

Model	Claimed	Effective	4K	128K
Gemini-1.5-Pro	1M	>128K	96.7	94.4
GPT-4	128K	64K	96.6	81.2
Llama3.1 (70B)	128K	64K	96.5	66.6
Llama3.1 (8B)	128K	32K	95.5	77.0
Yi (34B)	200K	32K	93.3	77.3
GradientAI/Llama3 (70B)	1M	16K	95.1	72.1
LWM (7B)	1M	<4K	82.3	65.0

Mean accuracy (%) over the 13 tasks at 4K and 128K tokens.
LWM (7B) starts below the pass mark at 4K, so its effective length is under 4K although it degrades slowly.

Hsieh et al., "RULER: What's the Real Context Size of Your Long-Context Language Models?" COLM 2024, [arXiv:2404.06654v3](https://arxiv.org/abs/2404.06654v3), Sections 1 and 4, Table 3.

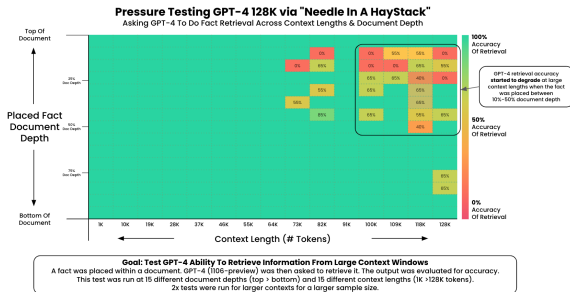
- ▶ Liu et al. measured the edge bias under controlled conditions. Each prompt holds p Wikipedia passages of at most 100 tokens: one answers a NaturalQuestions-Open question, and the other $p - 1$ are the passages a retriever ranks highest among those without the answer. Only the position of the answering passage changes.
- ▶ GPT-3.5-Turbo with $p = 20$ is right 75.8% of the time with the passage first, 63.2% with it last, and 53.8% at position 10: a U-shaped curve of primacy and recency.
- ▶ With no passages (closed book) it scores 56.1%; with only the answering passage, 88.3%. At position 10 it does worse than with no passages at all.
- ▶ A longer window does not change the curve. GPT-3.5-Turbo (16K) and Claude-1.3 (100K) score almost exactly like their 4K and 8K versions on inputs that fit in both.
- ▶ What to report: accuracy as a function of position, and the gap between the best and the worst position. An average over positions hides the middle.



Accuracy against the position of the answering passage; the dashed line is closed-book accuracy. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” TACL 2024, [arXiv:2307.03172v3](https://arxiv.org/abs/2307.03172v3), Figure 1; numbers from Tables 1 and 6.

The Needle-in-a-Haystack Test

- ▶ **Needle in a haystack (NIAH)**, released by Greg Kamradt in November 2023: insert one sentence, the needle, at depth $x\%$ of a haystack of Paul Graham essays cut to n tokens, ask for it, and sweep x and n into a heatmap.
- ▶ Needle: "The best thing to do in San Francisco is eat a sandwich and sit in Dolores Park on a sunny day." Question: "What is the best thing to do in San Francisco?" ("most fun thing" in the code released in November 2023). An LLM judge scores the answer from 1 to 10.
- ▶ It tests literal retrieval of one fact. On GPT-4 128K, accuracy "started to degrade at large context lengths when the fact was placed between 10%–50% document depth".
- ▶ A pass is weak evidence: the question repeats the needle's words, there is one fact and nothing to combine, and recent models score near-perfectly.
- ▶ Models with "nearly perfect accuracy in the vanilla NIAH test" still show "large performance drops as the context length increases" on RULER. Across 59 models, HELMET finds that "synthetic tasks like NIAH do not reliably predict downstream performance."

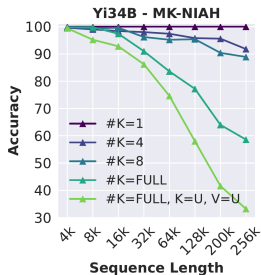
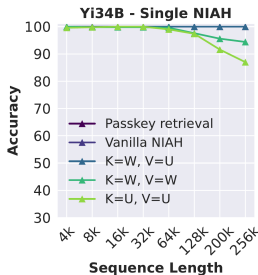


RULER: Thirteen Tasks Beyond One Needle

- Synthetic tasks with controllable length and difficulty: 500 examples per task at each length from 4K to 128K, and an answer counts when it contains the target. Four categories:

- **Retrieval:** NIAH with word, number or UUID (random 32-digit identifier) needles; extra needles with other keys as distractors; several values or several keys to return.
- **Multi-hop tracing:** in “VAR X1 = 12345 ... VAR X2 = X1 ... VAR X3 = X2”, name every variable that holds 12345.
- **Aggregation:** return the most frequent words of a long word list, a proxy for summarization.
- **Question answering:** the paragraphs that answer SQuAD and HotpotQA questions, hidden among other paragraphs of the same dataset.

- The 17 models tested all claim 32K or more, yet “only half of them can maintain satisfactory performance at the length of 32K” (10 of the 17 in Table 3).
- Yi-34B claims 200K and keeps 100% on passkey retrieval (a number hidden in repeated filler text) and vanilla NIAH up to 256K. UUID needles and distractor keys pull it down; with the haystack full of distractor needles it loses 40 points at 256K.

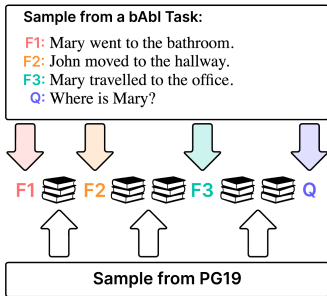


Yi-34B with different needle types (left; K and V give the key and value type: W words, U UUIDs) and with #K keys, one of them the target (right). Hsieh et al., [arXiv:2404.06654v3](https://arxiv.org/abs/2404.06654v3), Figure 2, first two of four panels.

- ▶ Start from a bAbI task (Weston et al., 2016): a short generated story about people, objects and places, followed by a question. Hide the story's sentences, in order, between sentences of books from the PG19 corpus until the sample is n tokens long.
- ▶ 20 tasks: fact chaining, induction, deduction, counting, lists and sets. QA1 needs one supporting fact, QA2 two and QA3 three; the story's other facts act as distractors. Ready-made splits go up to 10M tokens, and the generator extends to any length.
- ▶ Order matters. In the figure, F1 and F3 both mention Mary, and only the later one answers "Where is Mary?". Because the facts are generated, they cannot have leaked into pre-training data.
- ▶ Pass mark: 85% accuracy. Of 34 LLMs tested without fine-tuning on BABILong, most pass QA1 only up to 4K tokens and the best hold to 64K; with background text none reaches 85% on QA2, and the best QA3 score stays below 80%.
- ▶ The authors' summary: "popular LLMs effectively utilize only 10–20% of the context." BABILong is also used to test memory-based models.

a)

BABILong benchmark



Kuratov et al., "BABILong: Testing the Limits of LLMs with Long Context Reasoning-in-a-Haystack," NeurIPS 2024 Datasets and Benchmarks, [arXiv:2406.10149v2](https://arxiv.org/abs/2406.10149v2), Figure 1(a); numbers from the abstract, Table 1 and Section 3.1.

- ▶ The question shares no keyword with the needle; linking them takes one or two steps of world knowledge or common sense.
 - Needle: “Actually, Yuki lives next to the Semper Opera House.”
 - One hop: “Which character has been to Dresden?”
 - Two hops: “Which character has been to the state of Saxony?”
- ▶ Overlap as ROUGE-1 precision, the share of the question’s words that also occur in the needle: 0.905 for vanilla NIAH, 0.571 for RULER’s single needle, 0.553 for BABILong, 0.069 for NoLiMa.
- ▶ Haystacks are snippets of 10 open-license books, filtered so that no other sentence answers the question. 58 question–needle pairs, 26 needle positions, lengths from 250 to 32K tokens.
- ▶ The pass mark is relative: 85% of each model’s own score at 250 to 1K tokens. Of 13 models claiming at least 128K, 11 score at most half of their short-context score at 32K. GPT-4o falls from 99.3% to 69.7% and is effective only to 8K.

Llama 3.3 70B, accuracy (%) by context length

Question	8K	16K	32K
Direct (keyword in question)	98.3	98.5	98.5
One hop	84.1	73.2	56.2
Two hops	57.4	42.7	25.9

- ▶ Same needles and haystacks; only the question changes. With the needle’s keyword in the question, accuracy stays flat to 32K.
- ▶ For two-hop questions, accuracy falls with context length even when the needle stays in the last 2K tokens, at a fixed distance from the question, so position alone does not explain the drop.

Modarressi et al., “NoLiMa: Long-Context Evaluation Beyond Literal Matching,” ICML 2025, [arXiv:2502.05167v3](https://arxiv.org/abs/2502.05167v3), Sections 3 and 4, Tables 1, 3 and 6.

- ▶ Fu et al. take LLaMA-2 7B, pre-trained on 4K-token sequences, adjust the base that sets its RoPE rotation frequencies (RoPE rescaling) and continue pre-training the full model on 80K-token sequences in batches of 4M tokens.
- ▶ **Per-source length upsampling.** Keep each source's share of the SlimPajama corpus (67% CommonCrawl, 15% C4, 4.5% each for GitHub, Wikipedia and books, ...) and, inside every source, raise the share of documents longer than 4K from about 30% to about 70%. Upsampling long domains such as books instead, a common earlier practice, did worse and could hurt other domains.
- ▶ 500M tokens recover most of the needle accuracy over lengths 1K to 128K, though not yet beyond the 80K training length; about 5B saturate it at 88.0, against 87.1 for GPT-4-Turbo 128K. The run takes about 5 days on 8 A100 GPUs.
- ▶ LLaMA 3 also reached 128K by continued pre-training after an 8K main run.

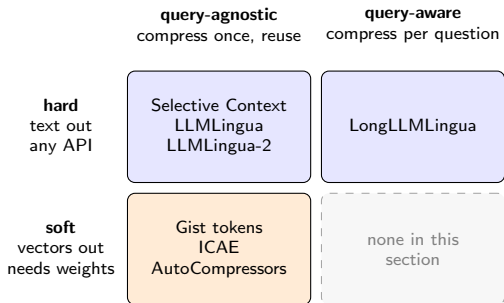
NIAH accuracy at up to 128K tokens

Tokens	100M	300M	500M	1B	5B	10B
Accuracy	37.8	59.0	81.1	85.3	88.0	84.0

- ▶ **The gap.** On a 128K-token book question-answering benchmark the same 7B model scores 27.4, ahead of the open baselines (24.3 and 26.3) and well behind GPT-4-Turbo (37.4).
- ▶ Llama 3.1 70B, a 128K model, is effective to 64K on RULER, 16K on BABILong QA1 and 2K on NoLiMa, each by its own pass mark.
- ▶ A few billion tokens buy a longer window. Each technique for making a model use that window, and making it affordable, should be judged with the tests above, at the length it targets.

Fu et al., "Data Engineering for Scaling Language Models to 128K Context," ICML 2024, proceedings.mlr.press/v235/fu24d, Sections 3–5, Figure 3, Tables 3 and 4. Effective lengths: RULER Table 3, BABILong Section 3.1, NoLiMa Table 3.

- ▶ Prompt compression shortens the input itself: fewer tokens, or a few vectors in place of many tokens. The model then runs as usual on the shorter input, so it stacks with the other techniques.
- ▶ The **compression ratio** is $n/\tilde{n}$, where n is the number of tokens in the original prompt and $\tilde{n}$ the number of tokens or vectors the LLM reads instead. LLMingua cuts a 2,366-token prompt to 117 tokens (next slide), a ratio of about $20\times$.
- ▶ **Hard** methods output text: here a subset of the original tokens, though paraphrasing methods also count as hard. A **black-box** model, reachable only through a text API, can read it, and a person can audit it.
- ▶ **Soft** methods output vectors the LLM reads in place of tokens, as input embeddings or as keys and values at every layer. They need the weights and some training.
- ▶ **Query-aware** methods see the question and keep what it needs; **query-agnostic** methods compress once and reuse the result for any question.



Two independent choices: what the compressor outputs, and whether it sees the question. Every soft method here compresses without the question, so its vectors can be computed once and cached.

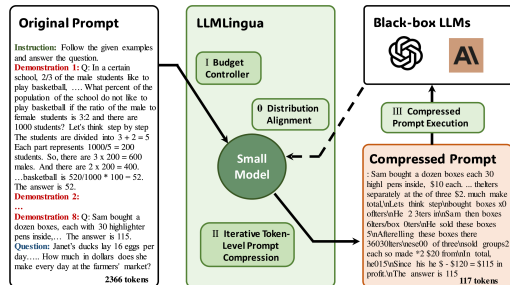
Hard/soft split after Li et al., “Prompt Compression for Large Language Models: A Survey,” NAACL 2025, aclanthology.org/2025.naacl-long.368.

- **Selective Context** scores each token with a small causal LM by its **self-information**

$$I(x_t) = -\log_2 p(x_t | x_{<t}),$$

where x_t is the t -th prompt token and p the small model's next-token probability. Predictable tokens carry little information. Perplexity is 2 raised to the mean of I .

- Token scores are summed over noun phrases (self-information is additive) and phrases below a chosen percentile are deleted. Deleting half cut per-token time from 110.8 to 76.3 ms in the paper's Vicuna-13B example. Averaged over all models, BERTScore, an embedding similarity to the full-context answer, fell by 0.023.
- **LLMLingua** adds three parts. A **budget controller** keeps 85% of the instruction and 90% of the question, and drops whole demonstrations first, lowest perplexity first. **Iterative token-level compression** scores 100-token segments given the text already compressed, because a deletion changes how surprising the next token is. **Distribution alignment** instruction-tunes the small LM on text generated by the target LLM.



GSM8K on GPT-3.5-Turbo: the 2,366-token chain-of-thought prompt shrinks to 117 tokens (20×) while exact match moves from 78.85 to 77.33. Selective Context reaches 44.20 at 15×.

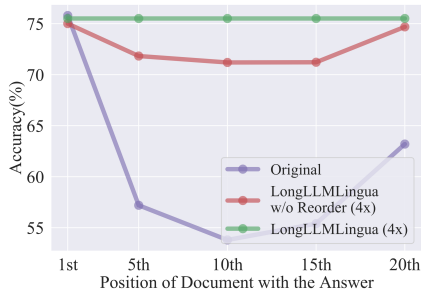
Jiang et al., "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," EMNLP 2023, [arXiv:2310.05736v2](https://arxiv.org/abs/2310.05736v2), Figure 1; numbers from Table 2. Selective Context: Li et al., "Compressing Context to Enhance Inference Efficiency of Large Language Models," EMNLP 2023, [arXiv:2310.06201v1](https://arxiv.org/abs/2310.06201v1).

- ▶ Question-agnostic deletion keeps whatever is surprising, relevant or not. On NaturalQuestions with 20 retrieved documents at $4\times$, LLMLingua scores 23.5 with the answer in document 10, below the 56.1 of GPT-3.5-Turbo given no documents.
- ▶ **Coarse step:** rank documents by the perplexity of the question given each document, with “We can get the answer to this question in the given documents” appended to the question. Lowest perplexity ranks first.
- ▶ **Fine step:** score each token by its **contrastive perplexity**

$$c_i = \text{ppl}(x_i \mid x_{<i}) - \text{ppl}(x_i \mid x^q, x_{<i}),$$

where x^q is the question and ppl the small LM’s perplexity of token x_i . Tokens that the question makes predictable score high.

- ▶ **Reordering** puts the top-ranked documents first, against the position bias, and top-ranked documents keep more tokens. **Subsequence recovery** swaps entity fragments the LLM copies into its answer for the full span in the original prompt.



Accuracy against the position of the answer document among 20 (GPT-3.5-Turbo, NaturalQuestions). With the answer in document 10: 54.1 on the full 2,946-token prompt, 71.2 at 748 tokens ($3.9\times$), 75.5 with reordering, at half the end-to-end latency.

Jiang et al., “LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression,” ACL 2024, [arXiv:2310.06839v2](https://arxiv.org/abs/2310.06839v2), Figure 1(b); numbers from Table 1.

- ▶ Self-information from a causal LM is a proxy: it sees only the left context, and nothing trains it to compress.
- ▶ **Data distillation:** GPT-4 compresses MeetingBank meeting transcripts in chunks of at most 512 tokens under strict rules: remove words only, keep their order, add nothing. As in knowledge distillation, a large model's outputs become the training labels of a small one.
- ▶ **Labels:** each original word is marked keep or drop by fuzzy matching against GPT-4's output, which tolerates changes of tense or plural. Samples with the 5% highest variation rate (words GPT-4 invented) or the 10% largest alignment gap are discarded.
- ▶ **Compressor:** a bidirectional encoder (XLM-RoBERTa-large, or multilingual BERT for the small version) with a linear keep/drop head, trained with cross-entropy. For a budget of $\tilde{n}$ words it keeps the $\tilde{n}$ words with the highest keep probability, in their original order.
- ▶ Trained on meetings only, it beats LLMLingua on LongBench, a suite of long-input tasks, under a 2,000-token budget (average 39.1 against 34.6) but trails the question-aware LongLLMLingua (48.0).

The same GSM8K demonstration under both compressors (start shown):

Original, 249 tokens: Sam bought a dozen boxes, each with 30 highlighter pens inside, for \$10 each box. He rearranged five of these boxes. . .

LLMLingua, 144 (117 in LLMLingua's Figure 1): Sam bought a dozen boxes each 30 highl pens inside, \$10 each. He reanged five of boxes. . .

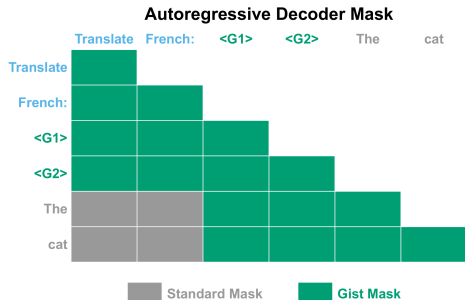
LLMLingua-2, 138: Sam bought dozen 30 highlighter pens \$10 rearranged five boxes into six highlighters. . .

Seconds on one V100 GPU (MeetingBank); rows 1–3 time the compressor alone. Uncompressed end to end: 14.9.

Compression ratio	2×	3×	5×
Selective Context	15.9	15.6	15.5
LLMLingua	2.9	2.1	1.5
LLMLingua-2	0.5	0.4	0.4
End to end, LLMLingua-2	9.4	7.5	5.2

Pan et al., "LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression," Findings of ACL 2024, [arXiv:2403.12968v2](https://arxiv.org/abs/2403.12968v2); Figure 12, Tables 2 and 5.

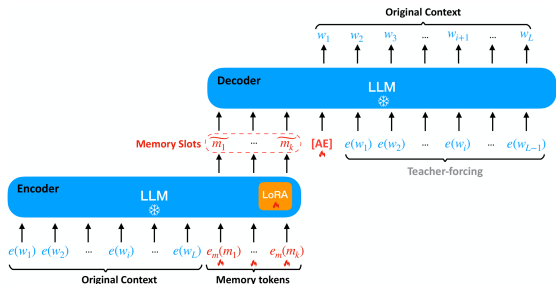
- ▶ **Gisting** inserts M copies of a new token $\langle G \rangle$ between the instruction and the input, and changes the attention mask so that tokens after the gist tokens cannot attend to anything before them. The instruction then reaches the output only through the gist tokens' activations, and ordinary instruction fine-tuning with this mask teaches the model to compress, at no extra training cost.
- ▶ The keys and values over the gist tokens act like soft prompts, but the model produces them from the prompt itself, in one forward pass and with no training per task. They are cached and reused for every input.
- ▶ LLaMA-7B and FLAN-T5-XXL with one gist token: human-written instructions of about 26 tokens become one ($26\times$). Judged by ChatGPT against the same model trained without the gist mask, the win rates are 45.8% and 42.5% (ties count half; 50% is parity).
- ▶ Against encoding the full prompt, a cached gist saves 40% of FLOPs and 4–7% of wall time. Against caching the full prompt's keys and values it saves little compute, but $26\times$ more prompts fit in the same memory.



Row tokens attend to column tokens. Grey cells are visible under the ordinary causal mask and hidden under the gist mask, so “The cat” sees the instruction only through $\langle G1 \rangle$ and $\langle G2 \rangle$.

Mu et al., “Learning to Compress Prompts with Gist Tokens,” NeurIPS 2023, [arXiv:2304.08467v3](https://arxiv.org/abs/2304.08467v3), Figure 2(a); numbers from Tables 1 and 3.

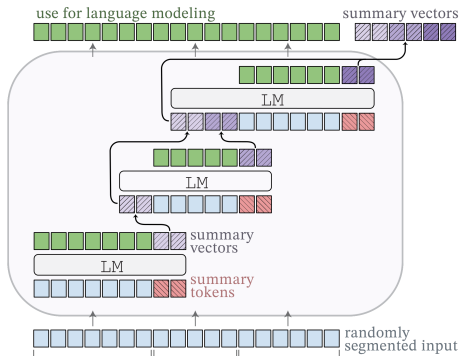
- **Encoder:** the LLM with a LoRA adapter reads the context followed by M learned memory tokens; its outputs at those positions are the M **memory slots**. **Decoder:** the same LLM, frozen, reads the slots in place of the context. About 1% extra parameters are trained.
- **Pretraining** on the Pile uses two self-supervised losses: autoencoding (after a special [AE] token, reproduce the context from the slots) and text continuation. **Fine-tuning** then uses PwC, 240k (context, prompt, response) triples: Pile texts with prompts and responses written by GPT-4.
- Default: 512 tokens into $M = 128$ slots ($4\times$). Ordinary text comes back almost exactly (BLEU 99.3) but random token strings do not (BLEU 0.2), so the slots hold what the LLM can rebuild from its own knowledge.
- Llama-2-7B-chat answering from 128 slots, judged by GPT-4: against the full context, 19.6% wins, 45.4% losses, 35.0% ties; against a 128-token summary written by GPT-4, 34.1% wins and 17.6% losses.
- For a batch of 8 inputs of 2,048 tokens, compressing and then decoding is $3.3\times$ faster than decoding from the full context.



Autoencoding pretraining: the frozen decoder rebuilds the context $w_1, \dots, w_L$ from the memory slots. The figure's k is M here, and its L counts context tokens.

Ge et al., "In-context Autoencoder for Context Compression in a Large Language Model," ICLR 2024, [arXiv:2307.06945v4](https://arxiv.org/abs/2307.06945v4), Figure 3; numbers from Tables 3, 4, 5 and 7.

- ▶ Split a long document into segments of s tokens and append 50 summary tokens to each segment. Their output states are the segment's **summary vectors**, a soft prompt for the text that follows.
- ▶ **Summary accumulation:** segment i is read with the summary vectors of all earlier segments prepended, so each new summary is computed from fresh text plus the older summaries.
- ▶ Training needs no labels. The loss is ordinary next-token prediction over the whole document, so a summary is good exactly when it helps predict later text. Segment lengths are randomised, and gradients stop after two segments to save memory.
- ▶ OPT-2.7B with $s = 2,048$ stores 40 tokens per summary vector. Fine-tuned on 30,720-token sequences, its perplexity on the last segment still falls as the compressed context grows from 14,336 to 28,672 tokens (11.21 to 11.18).
- ▶ Llama-2-7B: 4,096 tokens in 100 vectors predict about as well as 512 plain tokens (perplexity 5.08 against 5.06). Full attention over all 4,096 plain tokens is still better (4.80).
- ▶ Carrying a compressed state from one segment to the next is the recurrent-memory pattern.



Chevalier et al., "Adapting Language Models to Compress Contexts," EMNLP 2023, [arXiv:2305.14788v2](https://arxiv.org/abs/2305.14788v2), Figure 1; numbers from Tables 2 and 3.

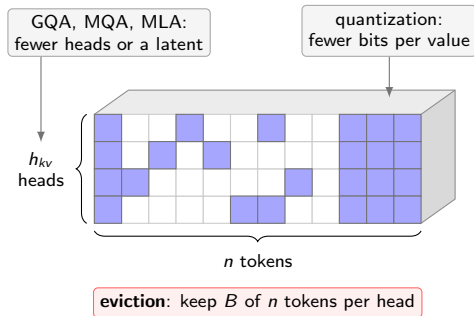
Method	Output	Needs	Question	Reported ratio	Reported effect
Selective Context	text	small LM	agnostic	about 2×	BERTScore −0.023
LLMLingua	text	small LM	agnostic	up to 20×	GSM8K 78.85 to 77.33
LongLLMLingua	text	small LM	aware	about 4×	NaturalQuestions 54.1 to 71.2
LLMLingua-2	text	small encoder	agnostic	2–5×	end to end 1.6–2.9× faster
Gist tokens	vectors	fine-tuned LLM	agnostic	26×	40% fewer FLOPs
ICAE	vectors	LoRA encoder	agnostic	4×	3.3× faster
AutoCompressors	vectors	fine-tuned LLM	agnostic	about 40×	4,096 tokens ≈ 512 plain

- ▶ **Exact strings go first.** LLMLingua’s 20× GSM8K prompt reads “highl pens” and “36030lters”, LongLLMLingua needs subsequence recovery to restore names, a gist model returned “Culture” for “Arts & Culture”, and ICAE rebuilds random token strings at BLEU 0.2. Spans that must survive verbatim, such as code, identifiers and numbers, are safer left uncompressed.
- ▶ **Hard or soft.** Hard output is readable, works with any API and needs no change to the target model; the reported operating points are about 20× on redundant few-shot prompts and 2–5× on documents. Soft output packs more into each position (ICAE’s 128 slots beat a 128-token GPT-4 summary), but it is tied to one model’s weights, costs a training run and cannot be read.
- ▶ **Aware or agnostic.** Seeing the question lifted accuracy at 4× from 23.5 to 71.2 on the same test (answer in document 10), but the compressed prompt must then be rebuilt for every question. Query-agnostic output is computed once and cached.

Each ratio is the paper’s own operating point on its own benchmark, so the rows are not directly comparable. Sources as on the previous slides.

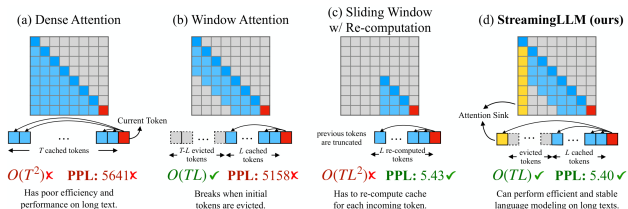
Three Ways to Shrink the KV Cache

- ▶ The cache size, $2 \cdot L \cdot h_{kv} \cdot d_h \cdot (\text{bytes per value}) \cdot n$, is a product, and each factor can be attacked separately.
- ▶ **Fewer numbers per token:** GQA and MQA share key/value heads, and MLA caches one low-rank latent per token.
- ▶ **Fewer bits per number:** KV-cache quantization such as TurboQuant.
- ▶ **Fewer tokens:** **eviction** keeps a budget of $B < n$ tokens per head and layer and drops the rest, so memory and the attention cost of each decoding step scale with B .
- ▶ Each eviction method fixes which tokens to keep, when to decide (once after the prompt, or at every decoding step), and from which signal (position or attention). An evicted entry cannot come back later.



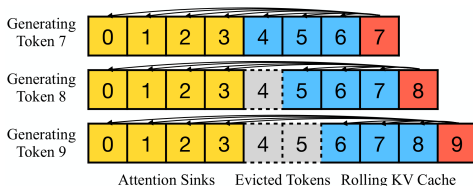
One layer's cache. Each of the 4 heads keeps its own $B = 6$ of the $n = 12$ tokens (blue); the grey boxes are the other two levers.

- ▶ **Window attention** at inference, on a model trained with full attention, caches only the last w tokens. It fails once the first tokens leave the cache: Llama-2-13B has perplexity 5158 with a 1024-token window, and 5.40 when 4 of the 1024 slots hold the first tokens (a 65K-token PG-19 book).
- ▶ **Attention sink**: above the bottom two layers, heads give the first tokens a large share of their attention whatever those tokens say (in Llama-2-7B at 4096 tokens, often more than half). Softmax weights must sum to one, so a head with nothing relevant to read still places its mass somewhere; the first tokens, visible to every later token in training, become that place.
- ▶ Evicting them removes a large part of the softmax denominator and shifts all remaining weights. The effect comes from position: with the first four tokens replaced by line breaks, perplexity is 5.60.



In the figure, T is the stream position and L the cache length (here not the layer count). Xiao et al., "Efficient Streaming Language Models with Attention Sinks," ICLR 2024, [arXiv:2309.17453v4](https://arxiv.org/abs/2309.17453v4), Figure 1; perplexities from Table 1, attention share from Appendix F.

- ▶ Keep the KV of the first 4 tokens (the sinks) and of a rolling window of recent tokens, and evict everything in between. The cache has the same size for any stream length, and no fine-tuning is needed.
- ▶ Positions are counted inside the cache. With tokens 0–3 and 6–8 cached, the token being generated gets position 7, its index in the cache, where the original text would give 9. Keys are cached before RoPE and rotated again at each step.
- ▶ Llama-2, MPT, Falcon and Pythia keep a stable perplexity over 4 million tokens. Four sinks suffice; in Llama-2-7B one or two are not enough. Per-token decoding is up to $22.2\times$ faster than a sliding window with recomputation (Llama-2-13B, one A6000 GPU).
- ▶ Pre-training with one learnable sink token at the start of every sample makes that token alone sufficient (160M-parameter models).



Xiao et al., [arXiv:2309.17453v4](https://arxiv.org/abs/2309.17453v4), Figure 4; numbers from Tables 2 and 3, Figures 5 and 10, and Appendices C and D.

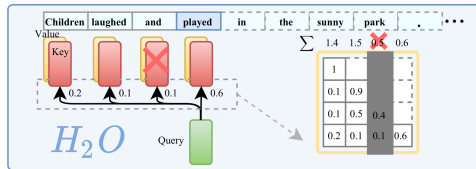
What it cannot do. An evicted token is gone, so StreamingLLM bounds memory without extending what the model can recall. On StreamEval (questions about lines seen earlier in the stream), accuracy drops to zero once the answer lies farther back than the cache. On LongBench, a cache of 4 sinks and 3496 recent tokens scores 11.6 on NarrativeQA, against 18.7 for keeping the first and last 1750 tokens of the input.

- ▶ Attention in OPT is over 95% sparse at inference, and the attention each token accumulates follows a power law: a few **heavy hitters** (H₂) collect most of it.
- ▶ **Policy:** half of B holds the newest tokens, half the tokens with the highest running score

$$C_j \leftarrow C_j + a_{t,j},$$

where $a_{t,j}$ is the softmax weight that the query of decoding step t puts on cached token j . Each step adds one token and evicts the lowest-scoring older one.

- ▶ A 20% cache (5× less KV memory) matches the full cache on most tasks for OPT, LLaMA and GPT-NeoX. On a 16 GB T4 GPU it allows larger batches and less CPU offloading: up to 29× the throughput of DeepSpeed ZeRO-Inference and Hugging Face Accelerate, and 3× that of FlexGen.

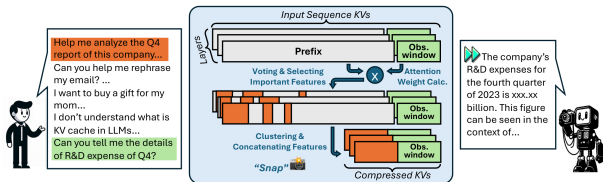


Column sums of past attention (Σ) decide eviction: “and” (0.5) leaves the cache. Zhang et al., “H₂O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models,” NeurIPS 2023, [arXiv:2306.14048v3](https://arxiv.org/abs/2306.14048v3), Figure 1 (lower left).

	COPA	OpenBookQA	PiQA	Winogrande
Full cache	85.00	43.20	78.51	70.24
Recent tokens only	48.00	25.20	55.82	49.17
Recent + H ₂	84.00	43.00	78.45	69.06

OPT-30B with a 20% cache, accuracy in %: Table 2 of the same paper; throughput from Tables 3 and 4.

- ▶ The prompt positions a head reads while generating are already visible in the attention of the last prompt tokens, and they stay the same over 512 generated tokens. Those last tokens usually contain the question.
- ▶ **Once, after prefill**, for each head: the last w_{obs} prompt tokens (16 or 32) vote $C_j = \sum_{i \in \text{obs}} a_{i,j}$ for every earlier position j , where $a_{i,j}$ is the attention weight of query i on key j . Max-pooling over C (kernel 5 or 7) keeps the neighbours of a strong position with it, and the top $B - w_{\text{obs}}$ positions plus the window form the cache. Generated tokens are appended without further eviction.
- ▶ LWM-Text-Chat-1M on one A100-80GB: at 16K tokens, decoding is $3.6\times$ faster and the longest input at batch size 2 grows $8.2\times$ (16K to 131K); with $B = 1024$, a planted sentence (the needle) is retrieved correctly up to 140K tokens, with a small accuracy drop up to 380K. On LongBench (about 13K tokens per input), Mistral-7B with $B = 1024$ beats H₂O with 4096 on 11 of 16 tasks.



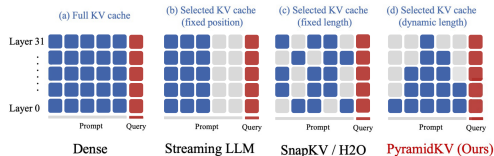
Li et al., “SnapKV: LLM Knows What You are Looking for Before Generation,” NeurIPS 2024, proceedings.neurips.cc, Figure 1; numbers from Sections 3 and 5.

- **Pyramidal information funneling:** in multi-document QA, LLaMA spreads attention almost uniformly in the lowest layers, keeps it inside each document in the middle layers, and concentrates it on a few tokens, sinks among them, in the upper layers.
- A uniform budget therefore wastes slots at the top and starves the bottom. PyramidKV keeps the last $\alpha = 8$ tokens in every layer and spreads the rest linearly over the L layers with average B :

$$B_\ell = B_0 - \frac{B_0 - B_{L-1}}{L - 1} \ell,$$

$$B_{L-1} = \frac{B}{\beta}, \quad B_0 = 2B - \frac{B}{\beta},$$

where B_ℓ is the budget of layer ℓ and β sets the slope. With $\beta = 20$, layer 0 keeps $39B/20$ tokens and the top layer $B/20$. Within a layer, tokens are picked as in SnapKV.



Blue: kept prompt entries in each layer; red: the query tokens at the end of the prompt, always kept. Cai et al., “PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling,” COLM 2025; figure from the preprint [arXiv:2406.02069v4](https://arxiv.org/abs/2406.02069v4), Figure 1.

- LongBench (LLaMA-3-8B and 70B Instruct, Mistral-7B): full-cache quality with 12% of the cache; with 0.7%, up to 20.5 points above the uniform-budget methods (TREC).

- ▶ Every method above decides from what it has already seen: position, or the attention of past queries. A token that only a later question needs is dropped before that question exists.

Failure	Measured effect	Source
Question arrives after compression	SnapKV compressing a shared context before the query is known (Llama-3.1-8B): semantic retrieval falls from 19.0 to 9.7, multi-task from 5.1 to 0.0	SCBench, Table 5
Follow-up turns on one cached context	Eviction methods do well only on the first request; methods that keep all n entries improve as requests accumulate	SCBench, Figure 3
Easy needles flatter eviction	H ₂ O at 4× compression (Llama-3-8B-Instruct) on passkey retrieval, recalling a number hidden in filler text: 100% for a 7-digit key, 35.0% for a 64-digit key; H ₂ O never evicts during prefill, so the first digits come free. 2-bit KIVI quantization: 100% to 91.0%	Yuan et al., Section 3.3
Kernel support	Accumulated scores need the attention matrix that FlashAttention never writes out; H ₂ O has no FlashAttention-compatible kernel	Yuan et al., Table 1 and Section 4

- ▶ Eviction is safest when the question is in the prompt and the context is used once. For shared, reused contexts, SCBench found that methods keeping all n entries degrade far less.

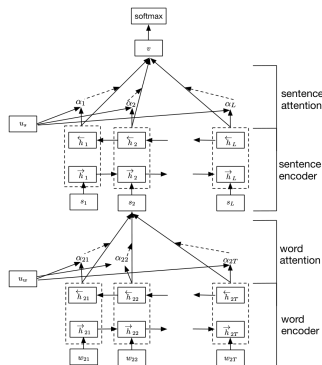
Li et al., “SCBench: A KV Cache-Centric Analysis of Long-Context Methods,” ICLR 2025, [arXiv:2412.10319v2](#); Yuan et al., “KV Cache Compression, But What Must We Give in Return? A Comprehensive Benchmark of Long Context Capable Approaches,” Findings of EMNLP 2024, [arXiv:2407.01527v2](#).

- ▶ A **hierarchical attention network** (HAN) builds sentence vectors from words, then a document vector from sentences.
- ▶ **Word level.** A bidirectional GRU reads sentence i and returns an annotation $\mathbf{h}_{it}$ for each word t . Attention pools the words of the sentence:

$$\alpha_{it} = \frac{\exp(\mathbf{u}_{it}^\top \mathbf{u}_w)}{\sum_{t'} \exp(\mathbf{u}_{it'}^\top \mathbf{u}_w)}, \quad \mathbf{s}_i = \sum_t \alpha_{it} \mathbf{h}_{it}.$$

Here $\mathbf{u}_{it} = \tanh(W_w \mathbf{h}_{it} + \mathbf{b}_w)$, $\mathbf{u}_w$ is a learned query ("what is the informative word"), α_{it} the weight of word t and $\mathbf{s}_i$ the sentence vector: additive attention with a learned query instead of a decoder state.

- ▶ **Sentence level.** A second bidirectional GRU reads $\mathbf{s}_1, \mathbf{s}_2, \dots$ and returns $\mathbf{h}_i$. The same pooling with its own query $\mathbf{u}_s$ gives the document vector $\mathbf{v} = \sum_i \alpha_i \mathbf{h}_i$, and $\text{softmax}(W_c \mathbf{v} + \mathbf{b}_c)$ predicts the class.
- ▶ Yelp 2015 accuracy (1.57M reviews, five ratings; Table 2): flat word-level CNN 61.5%, HAN with uniform weights ($\mathbf{u}_w = \mathbf{u}_s = \mathbf{0}$) 69.9%, full HAN 71.0%. The hierarchy gives most of the gain.

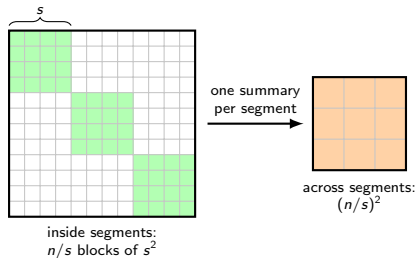


Yang et al., "Hierarchical Attention Networks for Document Classification," NAACL 2016, aclanthology.org/N16-1174, Figure 2.

- ▶ Replace the GRUs with Transformers. Split the n tokens into n/s segments of s tokens (s : segment length), attend fully inside each segment, then across one summary vector per segment. Scores per layer:

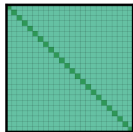
$$\frac{n}{s} \cdot s^2 + \left(\frac{n}{s}\right)^2 = ns + \left(\frac{n}{s}\right)^2 \quad \text{instead of } n^2.$$

- ▶ **Worked number:** $n = 16,384$ and $s = 128$ give $2,097,152 + 16,384 \approx 2.1$ million scores against $n^2 \approx 268$ million, a $127\times$ cut. The sum is smallest at $s = (2n)^{1/3}$, here $s = 32$ and 0.79 million scores.
- ▶ **Hi-Transformer** uses sentences as segments: a sentence Transformer over each sentence plus an appended [CLS] token, a document Transformer across the n/s [CLS] vectors, then a second sentence Transformer that re-reads each sentence with its document-aware vector. With sentences of s words and width d , the cost is $O(nsd + (n/s)^2 d)$ against $O(n^2 d)$.
- ▶ Words in different sentences meet only through summaries. Dropping the downward pass lowers accuracy on all three datasets tested, e.g. MIND news topics from 82.51% to 81.92% (Figure 2).

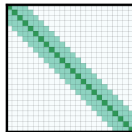


Shaded cells are the scores one layer computes for 12 tokens in segments of 4. Full attention would fill the whole square.

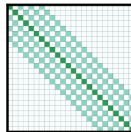
Wu et al., "Hi-Transformer: Hierarchical Interactive Transformer for Efficient and Effective Long Document Modeling," ACL 2021, [arXiv:2106.01040v3](https://arxiv.org/abs/2106.01040v3).



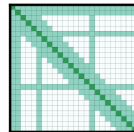
(a) Full n^2 attention



(b) Sliding window attention



(c) Dilated sliding window

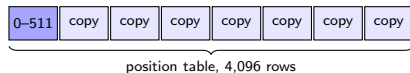


(d) Global+sliding window

Beltagy et al., "Longformer: The Long-Document Transformer," arXiv 2020, [arXiv:2004.05150v2](https://arxiv.org/abs/2004.05150v2), Figure 2.

- ▶ **(b) Sliding window**, here inside a bidirectional encoder: $w/2$ tokens on each side, cost $O(nw)$, receptive field $L \cdot w$ after L layers.
- ▶ **(c) Dilated window**: attend to every δ -th position, as a dilated convolution does. The receptive field grows to $L \cdot \delta \cdot w$ at the same cost. Heads can differ: undilated heads keep local detail while dilated heads reach far. The character-level model widens w from lower to upper layers and dilates only 2 heads of the upper layers.
- ▶ **(d) Global attention** on a few positions chosen per task: [CLS] for classification, every question token for QA. It is symmetric: a global token attends to all n tokens and all tokens attend to it. With g global tokens there are about $n(w + 2g)$ scores, still linear in n .
- ▶ Global attention has its own projections Q_g, K_g, V_g , initialised as copies of the window projections Q_s, K_s, V_s . WikiHop dev accuracy after 5 fine-tuning epochs (Table 10): 73.8 with both, 72.2 without the separate projections, 65.5 with no global attention.

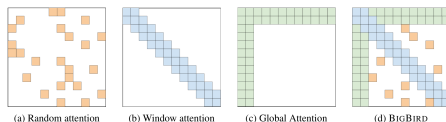
- ▶ Longformer starts from the RoBERTa checkpoint and continues masked-language-model (MLM) training. Every layer uses a window of $w = 512$, so each token does the same attention work as in 512-token RoBERTa.
- ▶ RoBERTa learned only 512 absolute position embeddings; all 4,096 positions start as 8 back-to-back copies of them. Several BERT heads attend mainly to the previous or next token, so the copies keep that local structure except at the 7 seams. LED, the encoder-decoder variant, stretches BART from 1K to 16K positions the same way.
- ▶ MLM bits per character, base size (Table 5): RoBERTa at 512 tokens 1.846; Longformer at 4,096 with random new positions 10.299, with copies 1.957, after 2K updates 1.753, after 65K updates 1.705.
- ▶ The RoBERTa baseline encodes 512-token pieces separately and concatenates the outputs. Longformer reads up to 4,096 tokens in one pass, with global attention on the question or on [CLS]. The largest gain is on Hyperpartisan: 645 news articles averaging 705 wordpieces.



Initial position table: RoBERTa's 512 learned rows, then 7 copies.

Task (metric)	RoBERTa	Longformer
WikiHop (acc.)	72.4	75.0
TriviaQA (F1)	74.3	75.2
HotpotQA (joint F1)	63.5	64.4
OntoNotes (avg. F1)	78.4	78.6
IMDB (acc.)	95.3	95.7
Hyperpartisan (F1)	87.4	94.8

Base models, development sets. Beltagy et al.,
[arXiv:2004.05150v2](#), Tables 5–7.

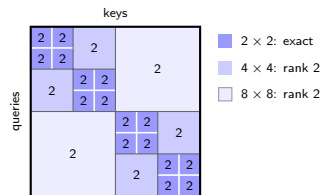


Zaheer et al., “Big Bird: Transformers for Longer Sequences,” NeurIPS 2020, [arXiv:2007.14062v2](https://arxiv.org/abs/2007.14062v2), Figure 1.

- ▶ As a graph, query i reads key j only along an edge, and full attention is the complete graph. BigBird keeps (a) r random keys per query, (b) a window of w neighbours, (c) g global tokens joined to every token both ways.
- ▶ Random graphs with $\tilde{O}(n)$ edges ($\tilde{O}$ hides log factors) have $O(\log n)$ shortest paths, so information mixes fast.
- ▶ BigBird-ITC (internal transformer construction: some existing tokens are made global) base uses blocks of $b = 64$ with $g = 2b$, $w = 3b$, $r = 3b$: each non-global query reads $128 + 192 + 192 = 512$ keys, as in 512-token BERT, but $n = 4,096$ (Table 8), $8\times$ longer on similar 16 GB chips.
- ▶ **Universal approximation** (Theorem 1): with a star in the graph, one global token joined to all, sparse attention approximates any continuous sequence-to-sequence function. A sparse encoder-decoder is Turing complete, assuming arbitrary precision as for full attention.
- ▶ **No free lunch** (Proposition 1): finding each vector's farthest vector takes one full-attention layer, but $\tilde{\Omega}(n^{1-o(1)})$ layers for any sparse pattern with $\tilde{O}(n)$ edges.
- ▶ Without global tokens, at 512 tokens: SQuAD 85.1 against 88.5 for BERT (Table 1). At 4,096 tokens: TriviaQA dev F1 79.5, against 75.2 for Longformer and 74.3 for RoBERTa (Table 2).

- ▶ An LSTM language model uses about 200 tokens of context but ignores word order beyond the last 50, keeping the distant past “only as a rough semantic field or topic.”
- ▶ H-Transformer-1D builds this into attention with a **hierarchical matrix** (H-matrix) from numerical analysis: exact blocks near the diagonal, low-rank blocks farther away, larger with distance.
- ▶ Average adjacent pairs of queries and keys (sum the values) to get $n/2, n/4, \dots$ coarse tokens over $\log_2(n/b)$ levels. At each level a block of b tokens attends only to neighbouring blocks, so a level- l score stands for $2^l \times 2^l$ token pairs. Every block is $b \times b$, which suits dense GPU kernels; building and applying the matrix each take about $5nbd$ operations, linear in n .
- ▶ One Billion Word perplexity: 20.25 at 144M parameters, against 21.8 for Transformer-XL at 800M (Table 2).

Khandelwal et al., “Sharp Nearby, Fuzzy Far Away: How Neural Language Models Use Context,” ACL 2018, [arXiv:1805.04623v1](#).



Ranks for 16 tokens, redrawn after Zhu & Soricut, “H-Transformer-1D: Fast One-Dimensional Hierarchical Attention for Sequences,” ACL 2021, [arXiv:2107.11906v1](#), Eq. (20).

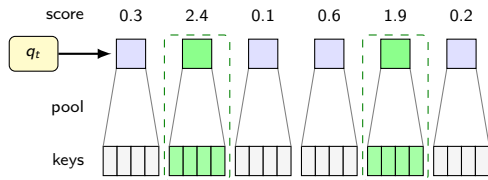
LRA accuracy	ListOps	Text	Pathfinder	Avg.
Transformer	36.37	64.27	71.40	54.39
BigBird	36.05	64.02	74.87	55.01
H-Transformer-1D	49.53	78.69	68.78	61.41

Long Range Arena (LRA): six classification tasks on 1K–16K tokens, such as ListOps (nested arithmetic) and Pathfinder (do two circles connect?). Avg. covers five tasks, two not shown. Every model here fails Path-X, which S4 later solved. Table 1.

Pattern	Who attends to whom	Scores per layer	What is lost	Examples
Sliding window	each token reads its w nearest tokens	nw	links beyond w need many layers	Mistral 7B, Gemma 2/3 local layers
Swin windows	local windows, shifted in alternate layers	linear in patches	far patches interact only over depth	Swin Transformer
Two-level segments	inside each segment, then across summaries	$ns + (n/s)^2$	token links across segments	HAN, HIBERT, Hi-Transformer
Window, dilation, global	window; dilated heads; g global tokens	$\approx n(w + 2g)$	global positions chosen by hand per task	Longformer, LED
Window, random, global	window; r random keys; g global tokens	$\approx n(w + r + 2g)$	some tasks need far more layers	BigBird
Pooled hierarchy (H-matrix)	exact near blocks; coarser blocks farther away	$\approx 5nb$	far detail averaged away	H-Transformer-1D
Byte patches	local byte models; a global model over patches	quadratic in patches	detail inside a patch compressed	MEGABYTE (fixed stride), BLT

- In every row, the keys a query may read are fixed before the query is compared with any key. Block-sparse attention lets each query choose which blocks to read.

- ▶ Split the n cached keys into n/b contiguous blocks of b tokens. Give each block one cheap summary key: the mean of its keys, a trained landmark token, or a learned compressed token.
- ▶ The query q_t scores the summaries, keeps the top- k blocks and runs exact softmax attention over their kb keys. Unlike the fixed patterns, the choice depends on the query.
- ▶ Keys read per query: $n/b + kb$ instead of n . The sum is smallest at $b = \sqrt{n/k}$, where it equals $2\sqrt{nk}$. Over all n queries: $n^2/b + nkb$ instead of n^2 .
- ▶ Contiguous blocks suit GPUs. A chosen block is one tile of the KV cache, copied from HBM to SRAM in one pass and multiplied on tensor cores like a FlashAttention tile. Scattered single tokens turn every load into a gather.



$n = 24$ keys, $b = 4$, $k = 2$: the query scores 6 summaries and reads the 8 keys of the green blocks exactly, 14 reads instead of 24.

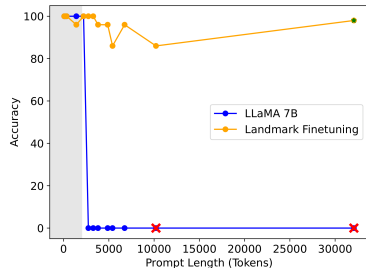
- ▶ **Worked number:** $n = 65,536$ and $k = 16$ give $b = 64$, so 1,024 summaries plus 1,024 exact keys: 2,048 reads instead of 65,536, a $32\times$ cut.

- ▶ Mohtashami and Jaggi (EPFL) append a **landmark token** to every block of b tokens. Training makes its key the summary of the block, so attention itself does the retrieval.
- ▶ **Grouped softmax.** Query token i takes one softmax over its own block and all other landmarks, and a separate softmax inside each other block. A token j in another block gets the weight

$$a_{ij} = P_{ij} P_{i,p_j},$$

its probability inside its block times that of its block's landmark p_j , so the landmark gates the whole block.

- ▶ **Inference.** Each token and head loads the top- k blocks by landmark score; only the landmarks must stay on the GPU. Blocks of 50 tokens cut attention work about 50-fold.
- ▶ **LLaMA 7B**, fine-tuned for 15,000 steps at context length 512, finds a hidden pass key in 98% of 50 prompts at 32,070 tokens. The base model fails just past its 2,048-token training length.



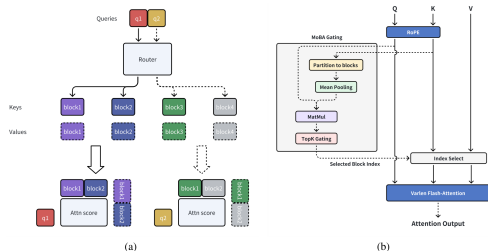
Pass-key accuracy against prompt length; grey band: LLaMA's training lengths, red crosses: out of memory, green star: cache offloaded to CPU, 5 blocks per head. Mohtashami & Jaggi, "Random-Access Infinite Context Length for Transformers," NeurIPS 2023, [arXiv:2305.16300v2](https://arxiv.org/abs/2305.16300v2), Figure 3(b).

- ▶ Lu et al. (Moonshot AI) apply mixture-of-experts (MoE) top- k gating to attention: blocks of keys and values are the experts.
- ▶ The gate has no parameters. Query t scores block i by

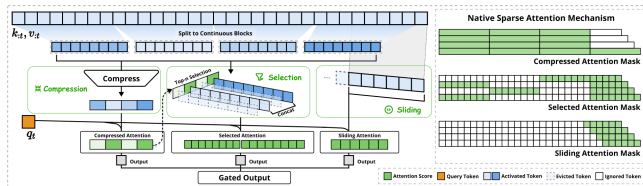
$$r_i = \langle q_t, \text{mean}(K_{[i]}) \rangle$$

($K_{[i]}$: the b keys of block i) and attends to the k highest-scoring blocks.

- ▶ Future blocks are excluded; the query's own block is always one of the k , with a causal mask, like a shared expert.
- ▶ Queries are regrouped by block for variable-length FlashAttention (panel b) and merged by online softmax.
- ▶ **Same parameters as full attention**, so layers can switch. A 1M-token Llama 3.1 8B trained 100B more tokens with MoBA ($b = 4096$, $k = 12$, last 3 of 32 layers dense; MoBA in prefill only) scores 0.7818 on RULER at 128K, full attention 0.7849.
- ▶ Attention runs $6.5\times$ faster than FlashAttention on a 1M-token prefill and $16\times$ at 10M. MoBA serves long-context requests in Moonshot's Kimi assistant.



(a) Two queries, four blocks, top-2 routing; (b) the gate in front of variable-length FlashAttention. Lu et al., “MoBA: Mixture of Block Attention for Long-Context LLMs,” 2025, [arXiv:2502.13189v1](https://arxiv.org/abs/2502.13189v1), Figure 1.



Left: the three branches; right: the masks they produce. Yuan et al., “Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention,” ACL 2025, aclanthology.org/2025.acl-long.1126, Figure 2.

- ▶ Yuan et al. (DeepSeek-AI, Peking University) give every query three branches, each with its own keys and values. In their 27B model:
 - **Compressed:** a small MLP maps each block of 32 keys (stride 16) to one coarse key.
 - **Selected:** 16 blocks of 64 tokens at full resolution, always including the first block and the 2 most recent.
 - **Sliding window:** the last $w = 512$ tokens, so fast-learned local patterns do not crowd out the other branches.

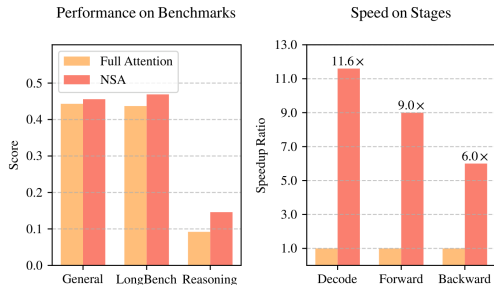
- ▶ A learned gate mixes the branches:

$$o_t = \sum_{c \in \{\text{cmp, slc, win}\}} g_t^c \text{Attn}(q_t, \tilde{K}_t^c, \tilde{V}_t^c),$$

where $\tilde{K}_t^c, \tilde{V}_t^c$ are the keys and values branch c gives query t , and $g_t^c \in [0, 1]$ is an MLP with a sigmoid on the token's features.

- ▶ Block scores come almost free: the compressed branch's probabilities $\text{softmax}(q_t^\top \tilde{K}_t^{\text{cmp}})$, summed over the heads of a GQA group.

- ▶ **Kernel.** A kernel written in Triton, a Python-based GPU programming language, loads queries by GQA group. The group shares one block list, so each chosen block moves from HBM to SRAM once and serves every head in one dense matrix multiply.
- ▶ **Training.** No extra loss: the compressed branch that scores blocks is trained by the language-model loss through the gate.
- ▶ **Setup.** A 27B-parameter MoE (3B active) with GQA, pretrained on 270B tokens at 8k and adapted to 32k with YaRN, next to a full-attention twin.
- ▶ **Quality.** General benchmarks 0.456 against 0.443 (ahead on 7 of 9), LongBench long-document tasks 0.469 against 0.437, perfect needle retrieval at 64k.
- ▶ **Speed at 64k on A100:** forward 9.0× and backward 6.0× over Triton FlashAttention-2. A decoding step reads $n/16 + 16 \cdot 64 + 512 = 5,632$ tokens instead of 65,536, an expected 11.6×.



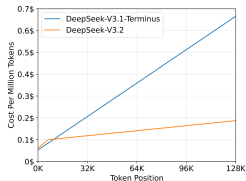
Left: average scores (“Reasoning”: AIME 2024 competition maths, after distillation); right: speedups at 64k. Yuan et al., [aclanthology.org/2025.acl-long.1126](https://arxiv.org/abs/2025.acl-long.1126), Figure 1.

- ▶ DeepSeek-V3.2 has one architectural change from DeepSeek-V3.1-Terminus: DeepSeek Sparse Attention (DSA), added by continued training.
- ▶ A **lightning indexer** scores each earlier token u for query token t :

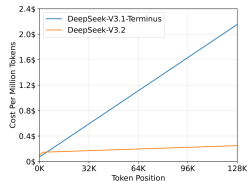
$$I_{t,u} = \sum_{j=1}^{H^I} \alpha_{t,j}^I \text{ReLU}(q_{t,j}^I \cdot k_u^I),$$

with H^I the small number of indexer heads, $q_{t,j}^I$ and the weight $\alpha_{t,j}^I$ from token t , and k_u^I from token u . It can run in 8-bit floating point (FP8).

- ▶ Each query keeps its $k = 2048$ top-scoring tokens and runs MLA on them in multi-query form, so one entry serves all heads. Core attention drops from $O(n^2)$ to $O(nk)$; the indexer stays $O(n^2)$ but is far cheaper.



(a) Prefilling



(b) Decoding

Cost per million tokens by position on H800 GPUs at 2 USD per GPU hour. At 128K, V3.1-Terminus against V3.2: \$0.67 vs \$0.19 (prefill), \$2.15 vs \$0.25 (decode). DeepSeek-AI, "DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models," 2025, [arXiv:2512.02556v1](https://arxiv.org/abs/2512.02556), Figure 3.

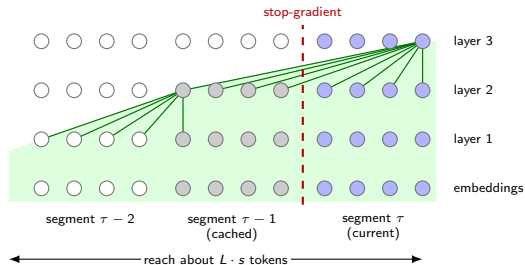
- ▶ **Warm-up:** attention stays dense; only the indexer trains, by a KL loss towards the head-summed attention. 1,000 steps, 2.1B tokens.
- ▶ **Sparse stage:** top- k on, all weights train. 15,000 steps, 943.7B tokens of 128K sequences.

Four Ways to Choose Where to Look

Method	Unit selected	Selection signal	Sparsity added	What the kernel needs
Landmark (2023)	blocks (50 tokens in its language-model runs)	attention to one landmark token per block	pretraining or fine-tuning	block list per token and head; cache can sit in CPU memory
MoBA (2025)	blocks of 256 to 4,096	q_t against mean-pooled keys; own block always kept	from scratch or continued; per-layer switch	queries regrouped by block, variable-length FlashAttention
NSA (2025)	blocks of 64, coarse blocks, a window	compressed-branch attention, summed over a GQA group	from the first pretraining step	one block list per GQA group, contiguous loads into SRAM
DSA (2025)	single tokens, $k = 2048$	lightning indexer: a few ReLU heads in FP8	continued: warm-up, then sparse stage	entries shared by all heads (multi-query MLA), not contiguous

- ▶ All four cut compute and memory traffic per query, while the KV cache still grows with n (Landmark can park it in CPU memory). The shared risk is a needed token outside the selected set; NSA's compressed branch and MoBA's dense last layers still see every position.
- ▶ Recurrent memory replaces the growing cache with a fixed-size state.

- ▶ Split n tokens into segments of s tokens and process them in order, carrying a memory of M vectors from one segment to the next. Attention then costs about $n(s + M)$ instead of n^2 .
- ▶ **Cached states.** Layer ℓ of segment τ also attends to the layer- $(\ell - 1)$ states that segment $\tau - 1$ left behind. A **stop-gradient** makes this cache a constant in the backward pass, so no gradient enters the previous segment.
- ▶ **Reach.** Each layer looks one segment further back, so L layers reach about $L \cdot s$ tokens.
- ▶ Sliding-window attention with window w also reaches $L \cdot w$ through depth, but runs over the whole sequence in one pass. Here only one segment and its cache are held at a time.



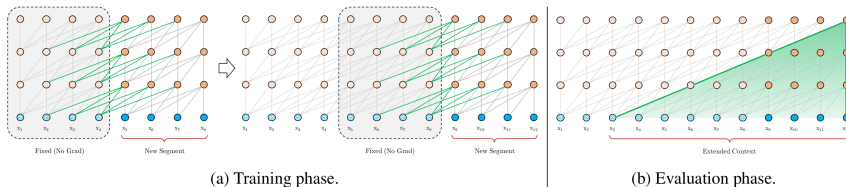
Here $s = 4$ and $L = 3$. The top state of the last token in segment τ reads layer 2 of segments $\tau - 1$ and τ ; the cached state it reaches in segment $\tau - 1$ had read layer 1 of segment $\tau - 2$. Shaded: everything that state can depend on.

- Transformer-XL (Dai et al., ACL 2019) feeds every layer the cached states of the previous segment:

$$\tilde{H}_\tau^{\ell-1} = [\text{SG}(H_{\tau-1}^{\ell-1}) \circ H_\tau^{\ell-1}], \quad H_\tau^\ell = \text{Layer}(H_\tau^{\ell-1} W_q, \tilde{H}_\tau^{\ell-1} W_k, \tilde{H}_\tau^{\ell-1} W_v).$$

$H_\tau^\ell \in \mathbb{R}^{s \times d}$ holds the layer- ℓ states of segment τ (d : model width; H^0 : token embeddings). SG is the stop-gradient and $\circ$ concatenates along the sequence, so the s queries of the segment meet $2s$ keys and values. W_q, W_k, W_v are the layer's projections; Layer is causal attention plus the feed-forward block. The cache can hold M states over several segments: $M = s$ in training, longer at evaluation.

- Evaluation.** A fixed-window model recomputes a whole window for every predicted token. Reusing the cache is up to **1,874 $\times$** faster per token on enwik8 at an attention length of 3,800.



Dai et al., "Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context," ACL 2019, [arXiv:1901.02860v3](https://arxiv.org/abs/1901.02860v3), Figure 2.

- **Absolute positions break.** Every segment reuses positions $1, \dots, s$, so token j of segment τ and token j of segment $\tau + 1$ get the same position vector, and the model cannot tell which came first.
- **Fix:** put the distance $i - j$ into every attention score, as RoPE and ALiBi also do. For query i and key j :

$$A_{i,j} = \underbrace{q_i^\top k_j}_{(a)} + \underbrace{q_i^\top W_{k,R} r_{i-j}}_{(b)} + \underbrace{u^\top k_j}_{(c)} + \underbrace{v^\top W_{k,R} r_{i-j}}_{(d)}$$

(a) content addressing, (b) a position term that depends on the query's content, (c) a global content bias, (d) a global position bias. q_i and k_j come from content alone, r_{i-j} is a fixed sinusoidal encoding of the distance, $W_{k,R}$ is its key projection, and $u, v \in \mathbb{R}^d$ are learned vectors shared by all query positions.

- Fixed sinusoids let a model trained with a short memory use a longer one: on WikiText-103, 384 tokens of attention in training and 1,600 at evaluation.

Model (WikiText-103)	RECL
Transformer-XL, 151M	900
QRNN	500
LSTM	400
Transformer-XL, 128M without recurrence	700 300
Vanilla Transformer	128

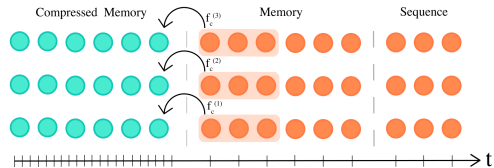
- **RECL** (relative effective context length, in words): grow the context until extra words cut perplexity by less than 1%, measured against the best model of the same group on the 10% hardest tokens. QRNN is a quasi-recurrent network.
- Hence the headline: 80% longer than RNNs and 450% longer than a vanilla Transformer.
- At publication: perplexity 18.3 on WikiText-103 (previous best 20.5) and 0.99 bits per character on enwik8.

Dai et al., [arXiv:1901.02860v3](https://arxiv.org/abs/1901.02860v3), Section 3.3, Tables 1, 2 and 8, Appendix C.

- ▶ Each layer keeps two first-in, first-out memories: the last M states, as in Transformer-XL, and M_c compressed states. The s states evicted by each new segment go through a compression function f_c that returns s/c compressed states (c : compression rate).
- ▶ Attention reads compressed memory, memory and segment together, so L layers reach $L(s + M + c M_c)$ tokens.
- ▶ f_c tried: pooling or 1D convolution with kernel and stride c , dilated convolution, or keeping the most-attended memories.
- ▶ f_c is trained locally, without backpropagation through time (BPTT) over long unrolls, by an **attention-reconstruction** loss:

$$\mathcal{L}_{\text{attn}} = \|\text{attn}(h, m) - \text{attn}(h, f_c(m))\|_2$$

h : the segment's states; m : the evicted memories;
 attn : the layer's content-only attention, frozen.
 What nothing attends to may be dropped. Best on Enwik8: a convolution with this loss, 0.973 bits per character (0.984 with an auto-encoding loss).



Three layers, $s = 3$, $M = M_c = 6$, $c = 3$. Rae et al., "Compressive Transformers for Long-Range Sequence Modelling," ICLR 2020, openreview.net/forum?id=SylKikSYDH, Figure 1.

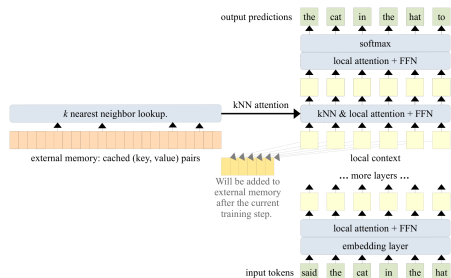
- ▶ **PG-19**, released with the model: 28,752 Project Gutenberg books published before 1919, 11 GB of text, 69K words per book on average against 3.6K for a WikiText-103 article.
- ▶ Test perplexity with 36 layers: 33.6 for the Compressive Transformer, 36.3 for Transformer-XL.

- ▶ Layer 9 of 12 is **kNN-augmented**. After each 512-token training step, its (key, value) pairs join an external memory that keeps the latest M pairs per head (up to 262K) and is emptied at each new document.
- ▶ Each query fetches its top $k = 32$ keys by approximate k -nearest-neighbour (kNN) search, as RAG retrievers do; about 90% of the true top k come back. A learned gate per head mixes the two results:

$$g = \sigma(b_g), \quad V_a = g \odot V_m + (1 - g) \odot V_c$$

V_a : the combined attention output; V_m : attention over the retrieved pairs; V_c : local attention; b_g : a learned scalar per head; σ : the sigmoid.

- ▶ **No gradient into the memory**, so stored pairs are reused across steps and M can grow. Old keys go stale as the weights change; normalizing keys and queries at least keeps old and new keys at the same magnitude.
- ▶ C4 web documents over 4K tokens: perplexity 17.20 without memory, 14.42 with an 8K memory. On arXiv math, an 8K memory matched a vanilla Transformer with $5\times$ the parameters.



Wu et al., "Memorizing Transformers," ICLR 2022, [arXiv:2203.08913v1](https://arxiv.org/abs/2203.08913v1), Figure 2.

- ▶ M memory tokens (real-valued vectors) go at both ends of every segment; X_τ holds the embeddings of its s tokens:

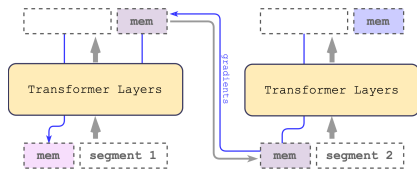
$$\tilde{X}_\tau = [H_\tau^{\text{mem}} \circ X_\tau \circ H_\tau^{\text{mem}}]$$

$$[H_\tau^{\text{read}} \circ H_\tau \circ H_\tau^{\text{write}}] = \text{Transformer}(\tilde{X}_\tau)$$

$$H_{\tau+1}^{\text{mem}} = H_\tau^{\text{write}}$$

H_τ : the segment's outputs. Under the causal mask the segment reads the start copy; the end copy sees the whole segment and becomes the next memory.

- ▶ The Transformer is unchanged: memory exists only in its input and output.
- ▶ Trained with BPTT over 0 to 4 previous segments: gradients reach earlier segments through the memory.
- ▶ WikiText-103, 150-token segments: 10 and 25 memory tokens give perplexity 25.04 and 24.85, close to 24.68 for Transformer-XL with 75 cached states per layer.



Bulatov, Kuratov & Burtsev, "Recurrent Memory Transformer," NeurIPS 2022, [arXiv:2207.06881v2](https://arxiv.org/abs/2207.06881v2), Figure 1.

	Transformer-XL	RMT
Carried	M states per layer, $M \cdot L$ in all	M vectors
Gradient	stopped at the segment	BPTT through the memory
Reach	about $L \cdot s$	not tied to depth
Model change	relative positions	none, extra tokens

- ▶ **Curriculum.** Bulatov et al. (AAAI 2024) give a pretrained BERT 10 memory tokens and 512-token segments and add one segment each time training converges. Trained on up to 7 segments, it retrieved stored facts across 4,096 segments (about 2 million tokens) in a constant 3.6 GB of GPU memory.
- ▶ **BABILong** (facts hidden in long distractor text). RMT and ARMT (associative RMT) on GPT-2 (137M) are fine-tuned on 32 segments (16K tokens); ARMT adds a per-layer associative matrix memory. Accuracy, % (* best model; ARMT mean 49.6):

Model	One supporting fact			Three supporting facts	
	128K	10M	50M	128K	10M
GPT-4 (few-shot)	24.0			4.0	
RMT (137M)	99.1	76.4	64.8*	42.9	21.0
ARMT (145M)	99.9	89.4	79.9*	86.0	37.0

- ▶ **Cost.** Segments run one after another, and BPTT stores every unrolled segment: Chevalier et al., building AutoCompressors on RMT, ran out of GPU memory fine-tuning a 2.7B RMT baseline on 30K tokens. A fixed memory also drops facts (three-fact columns).

Bulatov et al., “Beyond Attention: Breaking the Limits of Transformer Context Length with Recurrent Memory,” AAAI 2024, [doi:10.1609/aaai.v38i16.29722](https://doi.org/10.1609/aaai.v38i16.29722); Kuratov et al., “BABILong: Testing the Limits of LLMs with Long Context Reasoning-in-a-Haystack,” NeurIPS 2024, [arXiv:2406.10149v2](https://arxiv.org/abs/2406.10149v2); Rodkin et al., “Associative Recurrent Memory Transformer,” ICML 2024 Workshop, [arXiv:2407.04841v2](https://arxiv.org/abs/2407.04841v2), Table 1; Chevalier et al., “Adapting Language Models to Compress Contexts,” EMNLP 2023, [arXiv:2305.14788v2](https://arxiv.org/abs/2305.14788v2), Section 4.2.

- ▶ Causal softmax attention for token t of one head, with $q_t, k_t, v_t \in \mathbb{R}^{d_h}$:

$$o_t = \frac{\sum_{i \leq t} \exp(q_t^\top k_i / \sqrt{d_h}) v_i}{\sum_{j \leq t} \exp(q_t^\top k_j / \sqrt{d_h})}.$$

Step t reads all t cached keys and values.

- ▶ Katharopoulos et al. use a kernel $\phi(q)^\top \phi(k)$ instead, where ϕ maps a vector to positive features. Then $\phi(q_t)$ moves out of both sums:

$$S_t = S_{t-1} + \phi(k_t) v_t^\top, \quad z_t = z_{t-1} + \phi(k_t), \\ o_t = S_t^\top \phi(q_t) / z_t^\top \phi(q_t).$$

$S_t \in \mathbb{R}^{d_h \times d_h}$ sums the key-value outer products so far;
 $z_t \in \mathbb{R}^{d_h}$ sums the keys and normalises the read.

- ▶ The result is a recurrent network with a matrix state. Each step costs $O(d_h^2)$ time and memory for any t , and there is no KV cache. Training stays parallel, as cumulative sums over the known sequence.
- ▶ The paper's map is $\phi(x) = \text{elu}(x) + 1$, with elu the exponential linear unit: positive, and unlike ReLU it keeps a gradient for $x < 0$. Exact softmax is out of reach, because the exponential kernel needs infinitely many features.
- ▶ CIFAR-10 generated pixel by pixel, after one week of training: 17.85 images/s against 0.004 for softmax attention ($4,462\times$) and 0.32 for softmax with a KV cache (over $50\times$), at 3.40 against 3.47 bits/dim (negative log-likelihood in bits per pixel channel; lower is better).

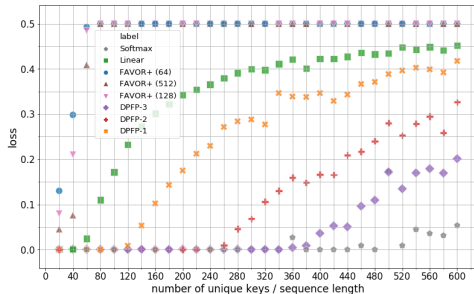
Katharopoulos et al., "Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention," ICML 2020, [arXiv:2006.16236v3](https://arxiv.org/abs/2006.16236v3), Section 3, Tables 2 and 4.

- Drop ϕ and the normaliser and store the transpose, as the delta-rule papers do. A write adds an outer product; a read multiplies by a query q :

$$S_t = S_{t-1} + v_t k_t^\top, \quad S_t q = \sum_{i \leq t} v_i (k_i^\top q).$$

Each stored value v_i comes back weighted by the match $k_i^\top q$.

- Example**, $d_h = 2$: store v_1 under $k_1 = (1, 0)^\top$ and v_2 under $k_2 = (0, 1)^\top$. Reading with k_1 returns v_1 . A third pair under $k_3 = (1, 1)^\top / \sqrt{2}$ turns that read into $v_1 + 0.71 v_3$.
- At most d_h keys can be orthogonal, so beyond d_h pairs exact recall is impossible. With $d_h = 64$, linear attention errs from about 60 pairs.
- Writing v'_1 under k_1 again makes the read return $v_1 + v'_1$: the memory can neither forget nor overwrite.



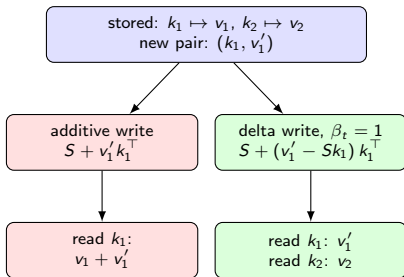
Retrieval loss against the number of stored pairs, $d_h = 64$; green: linear attention, grey: softmax attention. Schlag et al., "Linear Transformers Are Secretly Fast Weight Programmers," ICML 2021, [arXiv:2102.11174v3](https://arxiv.org/abs/2102.11174v3), Figure 2.

- Schlag et al. read the value stored under the incoming key, $\bar{v}_t = S_{t-1}k_t$, and replace it by the blend $\beta_t v_t + (1 - \beta_t) \bar{v}_t$:

$$\begin{aligned} S_t &= S_{t-1} + \beta_t (v_t - S_{t-1}k_t) k_t^\top \\ &= S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top. \end{aligned}$$

$\beta_t \in (0, 1)$ is a writing strength computed from the token. With unit-length keys, $\beta_t = 1$ replaces the old value exactly.

- Only the error $v_t - S_{t-1}k_t$ is written. The update is one gradient step, with learning rate β_t , on the loss $\frac{1}{2} \|S k_t - v_t\|^2$.
- The original algorithm ran token by token. Yang et al. made it chunkwise parallel and trained a 1.3B-parameter DeltaNet on 100B tokens.



The two-dimensional keys of the previous example. The delta write removes v_1 before adding v_1' . Since $k_2 \perp k_1$, neither write touches the pair under k_2 .

Schlag et al., [arXiv:2102.11174v3](#), Section 4.2; Yang et al., “Parallelizing Linear Transformers with the Delta Rule over Sequence Length,” NeurIPS 2024, [arXiv:2406.06484v6](#), Sections 2.2 and 3, Table 1.

- ▶ The delta rule edits one pair per step, so it cannot quickly clear the memory when the context changes. Mamba-2 can be written as linear attention with a scalar decay α_t that shrinks all pairs at once.
- ▶ Yang, Kautz & Hatamizadeh combine the two in the **gated delta rule**, with $\alpha_t \in (0, 1)$ computed from the token:

$$S_t = S_{t-1} \alpha_t (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

$\alpha_t \rightarrow 0$ empties the memory, and $\alpha_t \rightarrow 1$ gives back the pure delta rule.

Layer	State update	Forgets	Overwrites
Linear attention	$S_t = S_{t-1} + v_t k_t^\top$	no	no
Mamba-2	$S_t = \alpha_t S_{t-1} + v_t k_t^\top$	all pairs at once	no
DeltaNet	$S_t = S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$	no	the current key
Gated DeltaNet	$S_t = S_{t-1} \alpha_t (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$	all pairs at once	the current key

- ▶ At 1.3B parameters and 100B training tokens each, WikiText perplexity is 16.42 for Gated DeltaNet, 16.56 for Mamba-2, 17.71 for DeltaNet and 18.53 for a Llama-style Transformer.

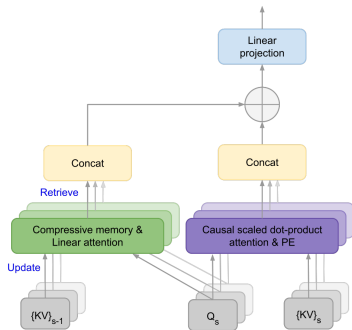
Yang, Kautz & Hatamizadeh, "Gated Delta Networks: Improving Mamba2 with Delta Rule," ICLR 2025, [arXiv:2412.06464v3](https://arxiv.org/abs/2412.06464v3), Table 1 (update rules) and Table 3.

- ▶ Each layer splits its input into segments of $s = 2,048$ tokens and runs causal softmax attention inside each one, giving a_t^{dot} .
- ▶ The same queries, keys and values also use one linear-attention memory per head ($\phi = \text{elu} + 1$). Segment j reads what earlier segments wrote and is written when it ends:

$$a_t^{\text{mem}} = \frac{S_{j-1} \phi(q_t)}{z_{j-1}^\top \phi(q_t)}, \quad S_j = S_{j-1} + \sum_{t \in j} v_t \phi(k_t)^\top.$$

A delta variant writes v_t minus the memory's current answer for k_t .

- ▶ A learned scalar per head mixes the two: $o_t = g a_t^{\text{mem}} + (1 - g) a_t^{\text{dot}}$, with $g = \text{sigmoid}(\gamma)$.
- ▶ The memory is $d_h^2 + d_h$ numbers per head and layer. Perplexity on PG19 books: 9.65, against 11.37 for Memorizing Transformers with a $114\times$ larger memory. A 1B model fine-tuned on 5K-token inputs finds a passkey hidden in 1M tokens; an 8B model set the BookSum (book summarisation) record at 500K.



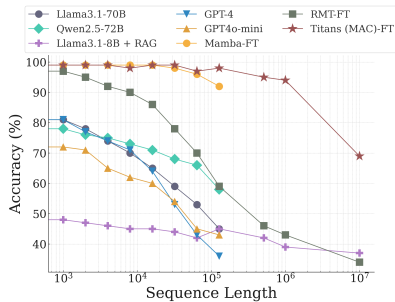
Munkhdalai et al., "Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention," preprint 2024, [arXiv:2404.07143v2](https://arxiv.org/abs/2404.07143v2), Figure 1; numbers from Tables 1–4.

- ▶ Behrouz et al. make the memory a small MLP $\mathcal{M}$, written by gradient steps on $\ell(\mathcal{M}_{t-1}; x_t) = \|\mathcal{M}_{t-1}(k_t) - v_t\|_2^2$. The gradient is the token's **surprise**. For a linear $\mathcal{M}$, one step is the delta rule.
- ▶ Surprise keeps momentum, and old content decays:

$$u_t = \eta_t u_{t-1} - \theta_t \nabla \ell(\mathcal{M}_{t-1}; x_t),$$
$$\mathcal{M}_t = (1 - \alpha_t) \mathcal{M}_{t-1} + u_t.$$

The momentum decay η_t , step size θ_t and forget rate $\alpha_t \in [0, 1]$ come from the token; $\alpha_t \rightarrow 1$ clears the memory, as $\alpha_t \rightarrow 0$ does in Gated DeltaNet. A read is a forward pass with no weight update.

- ▶ Memory as **context** prepends retrieved vectors and learned persistent tokens to each segment before attention; memory as a **gate** or a **layer** pairs it with sliding-window attention.
- ▶ Needle at 16K tokens (pass key / number / word): memory as context 98.4 / 97.4 / 95.2, DeltaNet 71.4 / 5.4 / 0.0, Mamba-2 5.4 / 0.0 / 0.0.



BABILong (questions about facts scattered through long texts), fine-tuned setting; red stars: Titans with memory as context. Behrouz et al., "Titans: Learning to Memorize at Test Time," preprint 2024, [arXiv:2501.00663v1](https://arxiv.org/abs/2501.00663v1), Figure 6(b); needle scores from Table 2.

- ▶ Every memory in this section has a fixed size, so the copying result applies.
- ▶ **Recall-heavy tasks** (table): Gated DeltaNet trails the Transformer by 6.4 points on average and by 28.5 on FDA. Attention layers whose 2K window spans these inputs close the gap.
- ▶ **Needles**: trained on 4K tokens, Gated DeltaNet finds a number hidden in essays 100.0% of the time at 1K tokens and 29.6% at 8K.
- ▶ **At scale**: from 410M to 7B parameters, MiniMax's lightning attention (a linear attention) matched softmax on common-sense tasks but fell short at needle retrieval, so its production model keeps one softmax layer in eight.
- ▶ **Independent check**: Hugging Face could not make Infini-attention reliable; quality fell as the number of memory compressions grew.

Accuracy (%), inputs truncated to 2K tokens

1.3B model	SWDE	FDA	Avg of 6
Mamba-2	19.1	25.3	29.8
DeltaNet	17.9	18.4	26.2
Gated DeltaNet	25.4	23.7	30.6
Llama-style Transformer	29.5	52.2	37.0
Gated DeltaNet + SWA	35.6	52.0	39.0

SWA: sliding-window attention layers, 2K window. SWDE: extract relations from web pages. FDA: retrieve key-value pairs from PDF documents. All models trained on the same 100B tokens. Yang, Kautz & Hatamizadeh, [arXiv:2412.06464v3](#), Table 4; needles from Table 2.

MiniMax, "MiniMax-01: Scaling Foundation Models with Lightning Attention," technical report 2025, [arXiv:2501.08313v1](#), Section 2.2 and Figure 7; Nguyen, von Werra & Wolf, "A Failed Experiment: Infini-Attention, and Why We Should Keep Trying?," Hugging Face blog, 14 Aug. 2024, [huggingface.co/blog](#).

Model	Linear layer	Full attention, ratio	Params (active)	Context
Qwen3-Next-80B-A3B, Sep. 2025	Gated DeltaNet	softmax with a sigmoid output gate; 3 : 1, 48 layers	80B (3B)	262,144; about 1M with YaRN
Kimi Linear, Oct. 2025	Kimi Delta Attention (Gated DeltaNet, per-channel decay)	MLA, no positional encoding; 3 : 1	48B (3B)	1M; KV cache up to 75% smaller
MiniMax-Text-01, Jan. 2025	lightning attention: linear, no delta rule	softmax with GQA; 7 : 1, 80 layers	456B (45.9B)	1M training, 4M inference

- ▶ Jamba keeps one attention layer in eight beside Mamba layers; Qwen3-Next and Kimi Linear keep one in four beside delta-rule memories.
- ▶ Kimi Linear at 1M tokens: 1.84 ms per output token against 11.48 ms for full MLA (over $6\times$, helped by larger batches; $2.2\times$ at batch size 1). Its ablation preferred 3:1 to 7:1, 15:1 or full attention alone.
- ▶ MiniMax returned to full attention for M2 (Oct. 2025): its hybrid had shown multi-hop reasoning deficits at scale, and other hidden weaknesses could not be ruled out.

Qwen3-Next model card, huggingface.co/Qwen; Qiu et al., “Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free,” preprint 2025, [arXiv:2505.06708v1](https://arxiv.org/abs/2505.06708v1); Kimi Team, “Kimi Linear: An Expressive, Efficient Attention Architecture,” technical report 2025, [arXiv:2510.26692v2](https://arxiv.org/abs/2510.26692v2); MiniMax, [arXiv:2501.08313v1](https://arxiv.org/abs/2501.08313v1); MiniMax, “Why Did MiniMax M2 End Up as a Full Attention Model?,” Hugging Face blog, 30 Oct. 2025, huggingface.co/blog.

Retrieval-augmented generation is a long-standing way around a short window. Two studies a year apart compared it with reading the whole input.

2023: Llama2-70B and a 43B GPT

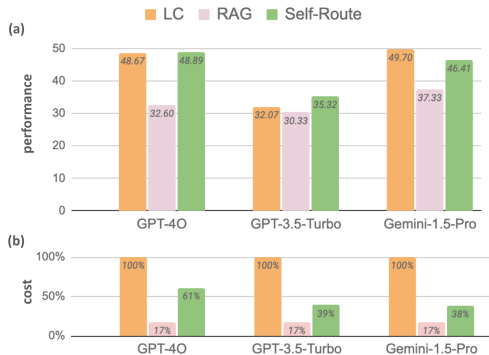
- ▶ A 4K window plus retrieval matched a window extended to 16K by position interpolation: average 36.02 vs 36.78 (Llama2-70B) and 29.32 vs 29.45 (GPT-43B), with much less computation.
- ▶ Retrieval still helped the long model: Llama2-70B-32k scored 40.9 without it and 43.6 with it, above GPT-3.5-turbo-16k (42.8), and generated 4× faster on NarrativeQA.
- ▶ The verdict moved because models got better at using long inputs. The cost gap did not move: most queries can be answered from a few retrieved chunks.

2024: Gemini-1.5-Pro, GPT-4o, GPT-3.5-Turbo

- ▶ With enough resources, the long-context (LC) model beat retrieval on average: 49.70 vs 37.33 for Gemini-1.5-Pro and 48.67 vs 32.60 for GPT-4o.
- ▶ Retrieval cost 17% of the long-context run, and for 63% of queries the two setups gave identical predictions, right and wrong alike.

Xu et al., “Retrieval Meets Long Context Large Language Models,” ICLR 2024, [arXiv:2310.03025v2](#), Section 1. Li et al., “Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach,” EMNLP 2024 Industry Track, [arXiv:2407.16833v2](#), Figure 1, Table 1 and Section 4.1.

- ▶ **Step 1, retrieve and route.** Give the model the query and the retrieved chunks (about 1.5k tokens) and allow it to reply “unanswerable”. Answerable queries stop here.
- ▶ **Step 2, read everything.** Only the queries declared unanswerable go to the long-context model with the full input (10k to 100k tokens).
- ▶ The router is the model’s own judgment of whether the chunks suffice, so no extra classifier is trained.
- ▶ Self-Route used 38% of the long-context cost with Gemini-1.5-Pro and 61% with GPT-4o, at average scores close to the long-context model (46.41 vs 49.70 and 48.89 vs 48.67).



“While long-context LLMs (LC) surpass RAG in long-context understanding, RAG is significantly more cost-efficient.” Li et al., [arXiv:2407.16833v2](https://arxiv.org/abs/2407.16833v2), Figure 1.

Family	What shrinks	Exact recall of an old token	Training	Typical failure
Hard prompt compression	tokens read	no, once dropped	small compressor	numbers and names dropped
Soft compression	tokens become vectors	approximate	encoder or LLM fine-tune	exact strings blur
KV-cache eviction	cache to B entries	no, once evicted	none	needle evicted before the question
Fixed sparse patterns	n^2 to about $n w$	through global tokens and depth	pretraining or adaptation	pattern ignores content
Learned block selection	n^2 to about $n^2/b + n k b$	yes, if its block is chosen	native or continued training	selection misses
Segment recurrence	memory per segment	no, beyond the memory	training through segments	sequential, slower training
Linear-attention memory	a fixed-size state	no, capacity bound	pretraining	recall-heavy tasks
Retrieval	tokens read	only what is retrieved	a retriever	retrieval misses, multi-hop

- Production systems combine rows: sparse or linear layers for most of the depth, a few full-attention layers for exact recall, quantized caches, and retrieval in front.

1. **Measure first.** Probe the model at your real input lengths with RULER- or NoLiMa-style tasks built from your own data. The effective length bounds what any later technique can deliver, whatever the advertised window.
2. **Knowledge in a large or changing corpus:** retrieve, and send the full context only when the retrieved chunks cannot answer, as Self-Route does.
3. **One long document that must be read whole, with serving memory as the limit:** compress the cache with quantization and query-aware eviction. Keep the full prompt when follow-up questions are unknown in advance.
4. **An unbounded stream:** sink tokens plus a recent window keep generation fluent; add a recurrent memory when old content must be recalled.
5. **Training a model:** build length in with block-sparse attention or delta-rule layers, mixed with a few full-attention layers.
6. **Re-measure after every change** on the same probe. Compression and sparsity fail silently, and they fail first on the long-range cases.

- ▶ **A long context costs compute, memory and quality.** Attention grows as $2n^2d$ FLOPs per layer and overtakes the weight matrices beyond $n = P/d$ (53K tokens for Llama 3.1 8B); the KV cache grows linearly, 128 KiB per token and 16 GiB per 128K-token request for the same model.
- ▶ **The usable window is shorter than the claimed one.** Lost in the middle gives a U-shaped accuracy curve over the position of the evidence; needle tests are weak evidence; RULER, BABILong and NoLiMa find effective lengths far below the advertised ones. A few billion tokens of continued pre-training extend the window, but using it is the hard part.
- ▶ **Prompt compression shrinks the input.** Hard methods delete tokens and work with any API (LLMLingua, 20× on a GSM8K prompt; LongLLMLingua adds the question; LLMLingua-2 classifies tokens). Soft methods turn the prompt into a few vectors (gist tokens, ICAE, AutoCompressors) and need the model's weights. Exact strings are the first casualty.
- ▶ **KV-cache eviction keeps B of the n cached tokens.** Attention sinks plus a recent window bound memory without extending recall; H₂O keeps heavy hitters, SnapKV lets the end of the prompt vote, PyramidKV gives lower layers more budget. Eviction fails when the question arrives after the decision or the context is reused.
- ▶ **Hierarchical attention reads near tokens finely and far tokens coarsely.** HAN pools words into sentences and sentences into documents; Longformer and BigBird combine local windows with global (and random) tokens at linear cost; H-Transformer-1D attends to distant blocks at coarser resolution. Fixed patterns choose positions without looking at the content.

- ▶ **Block-sparse attention lets the query choose.** Scoring block summaries and reading the top- k blocks costs about $n/b + kb$ keys per query. Landmark tokens, MoBA, NSA and DeepSeek Sparse Attention differ in the unit selected and the selection signal; NSA and DSA are trained with their sparsity: NSA matches or beats its full-attention twin, and DSA cuts the 128K decode cost from \$2.15 to \$0.25 per million tokens.
- ▶ **Segment recurrence carries a memory across segments.** Transformer-XL reuses the previous segment's states (reach about $L \cdot s$, relative positions); the Compressive Transformer compresses old states; Memorizing Transformers retrieve past keys by kNN; the RMT trains a few memory tokens through time and reaches millions of tokens on simple retrieval.
- ▶ **Linear attention turns memory into a fixed-size state.** The state S_t is an associative memory: additive writes interfere, the delta rule writes only the error, and Gated DeltaNet adds a decay. Infini-attention and Titans attach such memories to local attention. Fixed memories lose recall, so the 2025 hybrids keep one full-attention layer in four or eight.
- ▶ **Choose by measurement.** Frontier long-context models beat retrieval on average, but retrieval costs a fraction; routing between them (Self-Route) keeps most of the quality at 38–61% of the long-context cost. Measure effective length on your own data first, and re-measure after every compression or sparsity change.

1. Gemini Team, Google, “Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context,” 2024. [arXiv:2403.05530v5](#)
2. Llama Team, AI @ Meta, “The Llama 3 Herd of Models,” 2024. [arXiv:2407.21783v3](#) (source of the worked example’s configuration)
3. Hsieh et al., “RULER: What’s the Real Context Size of Your Long-Context Language Models?” COLM 2024. [arXiv:2404.06654v3](#) (source of Figure 2)
4. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” TACL 2024. [arXiv:2307.03172v3](#) (source of Figure 1)
5. G. Kamradt, `needle-in-a-haystack` repository (originally `LLMTest_NeedleInAHaystack`), 2023. [github.com/gkamradt/needle-in-a-haystack](#) (source of the GPT-4 128K heatmap)
6. Yen et al., “HELMET: How to Evaluate Long-Context Language Models Effectively and Thoroughly,” ICLR 2025. [arXiv:2410.02694v3](#)
7. Kuratov et al., “BABILong: Testing the Limits of LLMs with Long Context Reasoning-in-a-Haystack,” NeurIPS 2024 Datasets and Benchmarks. [arXiv:2406.10149v2](#) (source of Figure 1(a))
8. Weston et al., “Towards AI-Complete Question Answering: A Set of Prerequisite Toy Tasks,” ICLR 2016. [arXiv:1502.05698v10](#)

9. Modarressi et al., “NoLiMa: Long-Context Evaluation Beyond Literal Matching,” ICML 2025. [arXiv:2502.05167v3](#)
10. Fu et al., “Data Engineering for Scaling Language Models to 128K Context,” ICML 2024. [proceedings.mlr.press/v235/fu24d](#)
11. Jiang et al., “LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models,” EMNLP 2023. [arXiv:2310.05736v2](#) (source of Figure 1)
12. Li et al., “Compressing Context to Enhance Inference Efficiency of Large Language Models,” EMNLP 2023. [arXiv:2310.06201v1](#)
13. Pan et al., “LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression,” Findings of ACL 2024. [arXiv:2403.12968v2](#)
14. Jiang et al., “LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression,” ACL 2024. [arXiv:2310.06839v2](#) (source of Figure 1(b))
15. Mu et al., “Learning to Compress Prompts with Gist Tokens,” NeurIPS 2023. [arXiv:2304.08467v3](#) (source of Figure 2(a))
16. Ge et al., “In-context Autoencoder for Context Compression in a Large Language Model,” ICLR 2024. [arXiv:2307.06945v4](#) (source of Figure 3)

17. Chevalier et al., “Adapting Language Models to Compress Contexts,” EMNLP 2023. [arXiv:2305.14788v2](#) (source of Figure 1)
18. Li et al., “Prompt Compression for Large Language Models: A Survey,” NAACL 2025. [aclanthology.org/2025.naacl-long.368](#)
19. Bai et al., “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding,” ACL 2024. [arXiv:2308.14508v2](#)
20. Xiao et al., “Efficient Streaming Language Models with Attention Sinks,” ICLR 2024. [arXiv:2309.17453v4](#) (source of Figures 1 and 4)
21. Zhang et al., “H₂O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models,” NeurIPS 2023. [arXiv:2306.14048v3](#) (source of Figure 1)
22. Li et al., “SnapKV: LLM Knows What You are Looking for Before Generation,” NeurIPS 2024. [proceedings.neurips.cc](#) (source of Figure 1)
23. Cai et al., “PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling,” COLM 2025. [arXiv:2406.02069v4](#) (preprint; source of Figure 1)
24. Li et al., “SCBench: A KV Cache-Centric Analysis of Long-Context Methods,” ICLR 2025. [arXiv:2412.10319v2](#)

25. Liu et al., “KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache,” ICML 2024. [arXiv:2402.02750v2](#)
26. Yuan et al., “KV Cache Compression, But What Must We Give in Return? A Comprehensive Benchmark of Long Context Capable Approaches,” Findings of EMNLP 2024. [arXiv:2407.01527v2](#)
27. Yang et al., “Hierarchical Attention Networks for Document Classification,” NAACL 2016. [aclanthology.org/N16-1174](#) (source of Figure 2)
28. Wu et al., “Hi-Transformer: Hierarchical Interactive Transformer for Efficient and Effective Long Document Modeling,” ACL 2021. [arXiv:2106.01040v3](#)
29. Beltagy et al., “Longformer: The Long-Document Transformer,” 2020. [arXiv:2004.05150v2](#) (source of Figure 2)
30. Zaheer et al., “Big Bird: Transformers for Longer Sequences,” NeurIPS 2020. [arXiv:2007.14062v2](#) (source of Figure 1)
31. Zhu & Soricut, “H-Transformer-1D: Fast One-Dimensional Hierarchical Attention for Sequences,” ACL 2021. [arXiv:2107.11906v1](#) (source of the redrawn Eq. (20))
32. Khandelwal et al., “Sharp Nearby, Fuzzy Far Away: How Neural Language Models Use Context,” ACL 2018. [arXiv:1805.04623v1](#)

- 33. Zhang et al., “HIBERT: Document Level Pre-training of Hierarchical Bidirectional Transformers for Document Summarization,” ACL 2019. [arXiv:1905.06566v1](#)
- 34. Mohtashami & Jaggi, “Random-Access Infinite Context Length for Transformers,” NeurIPS 2023. [arXiv:2305.16300v2](#) (source of Figure 3(b))
- 35. Lu et al., “MoBA: Mixture of Block Attention for Long-Context LLMs,” 2025. [arXiv:2502.13189v1](#) (source of Figure 1)
- 36. Yuan et al., “Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention,” ACL 2025. [aclanthology.org/2025.acl-long.1126](#); preprint [arXiv:2502.11089v2](#) (source of Figures 1 and 2, ACL numbering)
- 37. DeepSeek-AI, “DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models,” 2025. [arXiv:2512.02556v1](#) (source of Figure 3)
- 38. Dai et al., “Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context,” ACL 2019. [arXiv:1901.02860v3](#) (source of Figure 2)
- 39. Rae et al., “Compressive Transformers for Long-Range Sequence Modelling,” ICLR 2020. [openreview.net/forum?id=SylKikSYDH](#) (source of Figure 1)
- 40. Wu et al., “Memorizing Transformers,” ICLR 2022. [arXiv:2203.08913v1](#) (source of Figure 2)

41. Bulatov, Kuratov & Burtsev, “Recurrent Memory Transformer,” NeurIPS 2022. [arXiv:2207.06881v2](#) (source of Figure 1)
42. Bulatov et al., “Beyond Attention: Breaking the Limits of Transformer Context Length with Recurrent Memory,” AAAI 2024. [doi:10.1609/aaai.v38i16.29722](#); extended version [arXiv:2304.11062v2](#)
43. Rodkin et al., “Associative Recurrent Memory Transformer,” ICML 2024 Workshop on Next Generation of Sequence Modeling Architectures. [arXiv:2407.04841v2](#)
44. Katharopoulos et al., “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” ICML 2020. [arXiv:2006.16236v3](#)
45. Schlag et al., “Linear Transformers Are Secretly Fast Weight Programmers,” ICML 2021. [arXiv:2102.11174v3](#) (source of Figure 2)
46. Yang et al., “Parallelizing Linear Transformers with the Delta Rule over Sequence Length,” NeurIPS 2024. [arXiv:2406.06484v6](#)
47. Yang, Kautz & Hatamizadeh, “Gated Delta Networks: Improving Mamba2 with Delta Rule,” ICLR 2025. [arXiv:2412.06464v3](#)
48. Munkhdalai et al., “Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention,” preprint, 2024. [arXiv:2404.07143v2](#) (source of Figure 1)

- 49. Behrouz, Zhong & Mirrokni, “Titans: Learning to Memorize at Test Time,” preprint, 2024.
[arXiv:2501.00663v1](#) (source of Figure 6(b))
- 50. MiniMax, “MiniMax-01: Scaling Foundation Models with Lightning Attention,” technical report, 2025.
[arXiv:2501.08313v1](#)
- 51. Nguyen, von Werra & Wolf, “A Failed Experiment: Infini-Attention, and Why We Should Keep Trying?,” Hugging Face blog, 14 Aug. 2024. [huggingface.co/blog/infini-attention](#)
- 52. Qwen Team, Qwen3-Next-80B-A3B-Instruct model card, Hugging Face, Sep. 2025.
[huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct](#)
- 53. Qiu et al., “Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free,” 2025. [arXiv:2505.06708v1](#)
- 54. Kimi Team, “Kimi Linear: An Expressive, Efficient Attention Architecture,” technical report, 2025.
[arXiv:2510.26692v2](#)
- 55. MiniMax, “Why Did MiniMax M2 End Up as a Full Attention Model?,” Hugging Face blog, 30 Oct. 2025.
[huggingface.co/blog/MiniMax-AI](#)
- 56. Xu et al., “Retrieval Meets Long Context Large Language Models,” ICLR 2024. [arXiv:2310.03025v2](#)
- 57. Li et al., “Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach,” EMNLP 2024 Industry Track. [arXiv:2407.16833v2](#) (source of Figure 1)